《AI Agents实战指南》

第一部分：基础入门（第1-3章）

第1章：AI Agent概述

1.1 什么是AI Agent

1.1.1 Agent的定义

AI Agent（人工智能代理） 是一种能够自主理解目标、规划任务路径、执行复杂操作并根据反馈迭代优化的智能系统。与传统的**大语言模型（LLM）对话**不同，Agent不仅能"回答问题"，更能"完成任务"。

简单来说：

- **LLM对话** = 你说一句话，它回复一段话
- **AI Agent** = 你说一个目标，它自主完成一系列操作并汇报结果

1.1.2 传统LLM对话 vs AI Agent

对比维度	传统LLM对话	AI Agent
交互模式	单轮问答	多轮自主规划
任务执行	仅生成文本	调用工具、执行动作
记忆能力	无状态（每次需重复上下文）	持久记忆、跨会话连续
自主性	完全依赖人类指令	目标导向、自主决策
外部交互	无法访问外部系统	可操控API、数据库、文件系统
错误处理	一次性回答	自我反思、迭代修正
适用场景	问答、写作、翻译	自动化流程、数据分析、助手应用

1.1.3 AI Agent的核心特征

一个典型的AI Agent具备以下核心特征：

1. **自主规划（Autonomous Planning）**：将复杂任务分解为可执行的子任务
2. **工具使用（Tool Use）**：调用外部API、操作文件系统、查询数据库
3. **记忆管理（Memory Management）**：维护短期上下文和长期知识
4. **持续迭代（Self-Election）**：根据执行结果反馈调整下一步行动
5. **目标导向（Goal-Oriented）**：围绕最终目标而非单纯响应输入

图1-1：AI Agent六层架构图

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                    用户交互层                                    │
│    ┌──────────────────────┐   ┌──────────────────────┐   ┌──────────────────────┐          │
│    │   Web界面    │  │   API接口    │  │   命令行CLI   │          │
│    └──────────────────────┘   └──────────────────────┘   └──────────────────────┘          │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                    Agent核心层                                    │
│    ┌──────────────────────────────────────────────────────────────────────────────────────┐          │
│    │  Profile/Persona    │  Planning/Feedback       │          │
│    │  ──────────────────────────────  │  ──────────────────────────────      │          │
│    │  • 角色定义          │  • 任务分解              │          │
│    │  • 行为规范          │  • 进度追踪              │          │
│    │  • 交互风格          │  • 自我修正              │          │
│    └──────────────────────────────────────────────────────────────────────────────────────┘          │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                    推理评估层                                    │
│    ┌──────────────────────┐   ┌──────────────────────┐   ┌──────────────────────┐        │
│    │  Reasoning   │  │  Evaluation  │  │  Reflection  │        │
│    │  推理引擎    │  │  结果评估    │  │  自我反思    │        │
│    └──────────────────────┘   └──────────────────────┘   └──────────────────────┘        │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                                    记忆知识层                                    │
│    ┌──────────────────────────────────────┐   ┌──────────────────────────────────────┐          │
│    │     Memory           │  │     Knowledge        │          │
│    │   ┌──────────────────────┐    │  │   ┌──────────────────────┐     │          │
│    │   │ 短期记忆      │    │  │   │ 向量数据库    │     │          │
```

```
│     │  │ (上下文窗口)  │     │   │   │ (结构化知识) │     │          │
│     │  └──────────────────────┘    │   │  └──────────────────────┘     │          │
│     │  ┌──────────────────────┐    │   │  ┌──────────────────────┐     │          │
│     │  │ 长期记忆       │     │   │   │ 知识图谱      │     │          │
│     │  │ (会话历史)     │     │   │   │ (关系网络)    │     │          │
│     │  └──────────────────────┘    │   │  └──────────────────────┘     │          │
│     └──────────────────────────────┘   └──────────────────────────────────┘          │
├──────────────────────────────────────────────────────────────────────────────────────┤
│                        动作工具层                                    │
│     ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │
│     │ 文件    │  │  API    │  │ 数据库 │  │ 搜索    │  │ 自定义 │ │
│     │ 操作    │  │ 调用    │  │ 查询    │  │ 引擎    │  │ 工具    │ │
│     └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  │
├──────────────────────────────────────────────────────────────────────────────────────┤
│                        LLM底座层                                    │
│     ┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐          │
│     │ GPT-4/4o    │   │ Claude 3     │   │ Gemini/LLaMA │           │
│     │ OpenAI     │   │ Anthropic   │   │ 开源模型     │          │
│     └──────────────────────┘  └──────────────────────┘  └──────────────────────┘          │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

架构说明:

- **用户交互层**: 负责接收用户指令，展示执行结果
- **Agent核心层**: 定义Agent的身份角色和规划控制逻辑
- **推理评估层**: 执行逻辑推理、评估输出质量、进行自我反思
- **记忆知识层**: 管理会话上下文、存储检索结构化知识
- **动作工具层**: 连接外部系统，提供实际执行能力
- **LLM底座层**: 提供基础语言理解和生成能力

1.2 Agent的四层演进

AI Agent的发展经历了四个层次的演进，从简单的问答交互到完全自主的行动系统。

图1-2：LLM交互四层次演进图

```
Level 1: 直接交互                          Level 2: 代理中介
┌──────────────────────────┐                    ┌──────────────────────────┐
│                          │                    │                          │
│   用户 ──→ LLM    │                    │   用户 ──→ Agent │
│                          │                    │         │        │
│   LLM ──→ 回答   │                    │         ↓        │
│                          │                    │       LLM        │
│   单一问答模式    │                    │         │        │
│   无状态         │                    │         ↓        │
│                          │                    │       回答        │
└──────────────────────────┘                    │   增加路由/过滤   │
                                                └──────────────────────────┘

Level 3: 工具调用                          Level 4: 完全自主
┌──────────────────────────┐                    ┌──────────────────────────┐
│                          │                    │                          │
│   用户 ──→ Agent │                    │   用户 ──→ Agent │
│         │        │                    │   (目标描述)    │
│         ↓        │                    │         │        │
│       LLM        │                    │         ↓        │
│      / | \       │                    │    自主规划      │
│     ↓  ↓  ↓      │                    │         │        │
│   搜索 API 文件  │                    │         ↓        │
│     \  |  /      │                    │    自主执行      │
│      结果整合    │                    │         │        │
│         ↓        │                    │         ↓        │
│       回答       │                    │    自我评估      │
│   工具增强能力   │                    │         │        │
└──────────────────────────┘                    │         ↓        │
                                                │    迭代优化      │
                                                │   目标达成       │
                                                └──────────────────────────┘
```

1.2.1 Level 1：直接交互（Direct Interaction）

特点：

- 用户直接与LLM对话
- 纯文本输入/输出
- 无状态，每次交互独立
- 典型应用：ChatGPT对话

局限性：

- 无法执行实际动作
- 无法访问实时信息
- 上下文有限

1.2.2 Level 2：代理中介（Agent Mediation）

新增能力：

- Agent作为用户与LLM之间的中介
- 增加路由逻辑（判断意图）
- 可进行内容过滤和增强
- 典型应用：客服机器人

增强点：

```
用户输入 → 意图识别 → 路由选择 → LLM处理 → 响应增强 → 用户
```

1.2.3 Level 3：工具调用（Tool Augmentation）

核心突破：

- LLM可以调用外部工具
- Function Calling / Tool Use
- 访问API、搜索、文件操作
- 典型应用：GPTs、插件系统

工作流程：

```
1. 用户提出需求
2. LLM判断需要调用的工具
3. 执行工具获取结果
4. 将结果纳入上下文
5. 生成最终回答
```

1.2.4 Level 4：完全自主（Full Autonomy）

最高形态：

- Agent自主规划任务路径
- 动态决策执行顺序
- 自我评估与迭代修正
- 典型应用：AutoGPT、BabyAGI

自主程度：

能力维度	Level 1	Level 2	Level 3	Level 4
任务分解	无	简单路由	静态分解	动态分解
工具调用	无	无	按需调用	自主选择
错误处理	无	简单反馈	预设重试	智能修正
自我评估	无	无	无	完整反思
多步协作	无	无	串行执行	并行/串行混合

1.3 Agent架构全景图

如前文的图1-1所示，AI Agent采用六层架构设计。每一层承担特定职责，层与层之间协同工作。

1.3.1 各层职责详解

第一层：用户交互层

用户交互层是Agent与用户之间的界面，负责：

- 接收用户指令（文本、语音、多模态）
- 格式化输出（文本、图表、操作结果）
- 管理会话状态（多轮对话连续性）

- 提供反馈机制（执行进度、结果确认）

第二层：Agent核心层

核心层定义Agent的"身份"和"大脑"：

- **Profile/Persona**：定义Agent的角色定位、性格特征、专业领域
- **Planning/Feedback**：任务规划引擎，负责将复杂任务分解为可执行步骤，并根据反馈调整计划

第三层：推理评估层

推理评估层是Agent的"思考能力"来源：

- **Reasoning**：逻辑推理引擎，处理复杂问题的思考过程
- **Evaluation**：结果评估模块，判断输出质量和任务完成度
- **Reflection**：自我反思机制，分析失败原因并生成改进策略

第四层：记忆知识层

记忆知识层赋予Agent"记忆"能力：

- **Memory（记忆）**：
  - 短期记忆：当前会话的上下文窗口
  - 长期记忆：跨会话积累的知识和偏好
- **Knowledge（知识）**：
  - 向量知识库：文档嵌入后的语义检索
  - 知识图谱：实体关系的结构化表示

第五层：动作工具层

动作工具层是Agent"动手做事"的能力：

- **基础工具**：文件读写、网络搜索、代码执行
- **API调用**：RESTful API、GraphQL、WebSocket
- **数据库**：SQL查询、向量检索、图数据库
- **自定义工具**：用户定义的业务逻辑接口

第六层：LLM底座层

LLM底座层提供基础语言智能：

- 文本理解与生成
- 意图识别与推理
- 上下文学习与指令跟随

1.3.2 层级间的数据流向

```
用户输入
    ↓
┌──────────────────────────────────────────┐
│ 用户交互层：解析指令，识别意图            │
└──────────────────────────────────────────┘
    ↓
┌──────────────────────────────────────────┐
│ Agent核心层：加载角色，确定任务目标       │
└──────────────────────────────────────────┘
    ↓
┌──────────────────────────────────────────┐
│ 推理评估层：制定执行计划                  │
└──────────────────────────────────────────┘
    ↓
┌──────────────────────────────────────────┐
│ 记忆知识层：检索相关记忆和知识            │
└──────────────────────────────────────────┘
    ↓
┌──────────────────────────────────────────┐
│ 动作工具层：调用工具执行具体操作          │
└──────────────────────────────────────────┘
    ↓
┌──────────────────────────────────────────┐
│ LLM底座层：生成响应，整合结果             │
└──────────────────────────────────────────┘
    ↓
┌──────────────────────────────────────────┐
│ 推理评估层：评估结果，反思修正            │
└──────────────────────────────────────────┘
```

```
    ↓
┌──────────────────────────────────────────────────────────────────────────┐
│ 用户交互层：呈现最终结果                                                  │
└──────────────────────────────────────────────────────────────────────────┘
    ↓
用户
```

1.4 单Agent五大核心组件

AI Agent的核心功能由五大组件协同完成，每个组件承担特定职责。

图1-3：单Agent五大组件关系图

```
                    ┌──────────────────────────────────────────────────────────┐
                    │          用户/环境输入                                    │
                    └──────────────────────────────────────────────────────────┘
                                        │
                                        ↓
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│                    Profile / Persona                                                             │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────┐   │
│   │  • 角色定义：我是什么类型的Agent？                                                        │   │
│   │  • 行为规范：我应该遵循什么规则？                                                         │   │
│   │  • 专业领域：我擅长处理什么任务？                                                         │   │
│   │  • 交互风格：我的沟通方式是怎样的？                                                       │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ↓
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│              Memory / Knowledge（记忆与知识）                                                    │
│   ┌──────────────────────────────────────┐      ┌──────────────────────────────────────┐      │
│   │       Memory                         │      │       Knowledge                      │      │
│   │   ┌──────────────────────────────┐   │      │   ┌──────────────────────────────┐   │      │
│   │   │  短期记忆                    │   │      │   │  向量数据库                  │   │      │
│   │   │  (上下文)                    │   │      │   │  (RAG知识)                   │   │      │
│   │   └──────────────────────────────┘   │      │   └──────────────────────────────┘   │      │
│   │   ┌──────────────────────────────┐   │      │   ┌──────────────────────────────┐   │      │
│   │   │  长期记忆                    │   │      │   │  知识图谱                    │   │      │
│   │   │  (历史积累)                  │   │      │   │  (关系网络)                  │   │      │
│   │   └──────────────────────────────┘   │      │   └──────────────────────────────┘   │      │
│   └──────────────────────────────────────┘      └──────────────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ↓
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│              Reasoning / Evaluation（推理与评估）                                                │
│   ┌──────────────────────────────────────┐      ┌──────────────────────────────────────┐      │
│   │      Reasoning                       │      │      Evaluation                      │      │
│   │   ┌──────────────────────────────┐   │      │   ┌──────────────────────────────┐   │      │
│   │   │  思维链(CoT)                 │   │      │   │  结果验证                    │   │      │
│   │   │  任务分解                    │   │      │   │  质量评估                    │   │      │
│   │   │  策略规划                    │   │      │   │  错误检测                    │   │      │
│   │   └──────────────────────────────┘   │      │   └──────────────────────────────┘   │      │
│   └──────────────────────────────────────┘      └──────────────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ↓
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│                Planning / Feedback                                                               │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────┐   │
│   │  • 任务分解：将目标拆解为可执行的子任务                                                   │   │
│   │  • 执行排序：确定任务执行的先后顺序                                                       │   │
│   │  • 进度追踪：监控子任务的完成状态                                                         │   │
│   │  • 动态调整：根据反馈重新规划路径                                                         │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                        │
                                        ↓
```

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                  Actions / Tools（动作与工具）                                                  │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐      │
│   │ 文件操作 │   │ API调用  │   │ 数据库   │   │ 代码执行 │      │
│   └──────────────────┘   └──────────────────┘   └──────────────────┘   └──────────────────┘      │
│   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐      │
│   │ 搜索查询 │   │ 定时任务 │   │ 消息推送 │   │ 自定义   │      │
│   └──────────────────┘   └──────────────────┘   └──────────────────┘   └──────────────────┘      │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ↓
                  ┌──────────────────────────────────────────────────────────┐
                  │           执行结果输出                                    │
                  └──────────────────────────────────────────────────────────┘
```

1.4.1 Profile / Persona（角色定义）

Profile定义了Agent的身份和行为准则：

```yaml
# Agent Profile 示例
profile:
  name: "文档助手小Doc"
  type: "专业文档助手"
  expertise:
    - 技术文档撰写
    - 代码注释生成
    - API文档编写
  behavior:
    - 表达清晰准确
    - 主动建议优化
    - 遵守格式规范
  constraints:
    - 不泄露敏感信息
    - 不生成有害内容
```

1.4.2 Actions / Tools（动作与工具）

Actions定义了Agent能执行的具体操作：

工具类型	功能描述	示例
文件操作	读写本地/云端文件	read_file, write_file
API调用	访问外部服务	http_request, api_call
数据库	查询和操作数据	sql_query, vector_search
代码执行	运行代码片段	python_eval, js_execute
搜索	检索信息	web_search, knowledge_retrieval

1.4.3 Memory / Knowledge（记忆与知识）

Memory（记忆）：

- **短期记忆**：维护当前会话的上下文，通常基于LLM的上下文窗口大小（如GPT-4的128K token）
- **长期记忆**：跨会话持久化存储用户偏好、历史交互、积累知识

Knowledge（知识）：

- **向量知识库**：通过RAG（检索增强生成）技术接入外部文档
- **知识图谱**：结构化的实体-关系知识网络，支持复杂推理

1.4.4 Reasoning / Evaluation（推理与评估）

Reasoning（推理）：

- **思维链（Chain of Thought）**：逐步推理，展示思考过程
- **任务分解**：将复杂任务拆解为可管理的子任务
- **策略规划**：制定最优执行策略

Evaluation（评估）：

- **结果验证**：检查输出是否符合预期
- **质量评估**：判断回答的准确性、完整性
- **错误检测**：识别执行过程中的问题

1.4.5 Planning / Feedback（规划与反馈）

Planning（规划）：

```
# 任务规划示例伪代码
def plan_task(user_goal):
    # 1. 理解目标
    objective = parse_goal(user_goal)

    # 2. 任务分解
    subtasks = decompose(objective)

    # 3. 依赖分析
    dependency_graph = analyze_dependencies(subtasks)

    # 4. 执行排序
    execution_order = topological_sort(dependency_graph)

    # 5. 资源规划
    required_tools = plan_resources(execution_order)

    return execution_plan
```

Feedback（反馈）：

- **执行反馈**：记录每个步骤的执行结果
- **动态调整**：根据反馈修正后续计划
- **自我反思**：分析成功与失败的原因

1.5 本书知识体系地图

图1-4：本书13大主题知识地图

```
                          ┌──────────────────────────────────────────────────────────┐
                          │        核心概念与基础            │
                          │  (第1-3章)                      │
                          │  • AI Agent概述 ← 你在这里      │
                          │  • 访问LLM                      │
                          │  • 构建第一个Agent              │
                          └──────────────────────────────────────────────────────────┘
                                        │
                                        ↓
┌──────────────────────────────────────────────────────────
```

```
│   └──────────────────┘   └──────────────────┘   └──────────────────┘                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

1.5.1 全书13大主题导览

章节	主题	核心内容	实战目标
1	AI Agent概述	六层架构、五大组件、发展演进	建立完整认知框架
2	访问LLM	API对比、参数调优、本地部署	掌握模型接入方法
3	构建第一个Agent	Assistants、Function Calling、文件处理	完成首个Agent开发
4	Prompt工程	系统Prompt、few-shot、输出控制	提升模型理解能力
5	Memory记忆管理	短期/长期记忆、向量存储、session管理	实现持久化交互
6	Tools工具设计	工具定义、API封装、Tool选择策略	扩展Agent执行能力
7	RAG检索增强	向量检索、知识库构建、混合检索	赋予专业领域知识
8	多Agent协作	角色分配、通信协议、任务编排	构建复杂Agent系统
9	安全与合规	数据隐私、内容安全、权限控制	确保Agent安全运行
10	评估与迭代	指标设计、在线测试、持续优化	建立质量保障体系
11	企业级架构	微服务、容器化、监控运维	构建生产级系统
12	行业应用案例	客服、代码助手、数据分析	启发业务场景思考
13	未来展望	AgentOS、多模态、自主AI	把握技术发展趋势

1.5.2 学习路径建议

路径一：快速入门（2天）

```
第1天：第1-3章（基础入门）
      ↓
第2天：选择一个方向深入（工具/RAG/多Agent）
```

路径二：系统学习（2周）

```
Week 1：第1-5章（基础 + 核心组件）
      ↓
Week 2：第6-10章（高级特性 + 质量保障）
```

路径三：实战精通（1个月）

```
第1-2周：第1-10章（理论基础）
      ↓
第3周：项目实战（跟随各章案例）
      ↓
第4周：第11-13章 + 自主项目
```

本章小结

本章介绍了AI Agent的核心概念和基础架构：

1. **AI Agent定义**：相比传统LLM对话，Agent具备自主规划、工具调用、记忆管理、持续迭代等核心能力
2. **四层演进**：从Level 1直接交互到Level 4完全自主，Agent能力逐步增强
3. **六层架构**：用户交互层、Agent核心层、推理评估层、记忆知识层、动作工具层、LLM底座层
4. **五大组件**：Profile、Actions/Tools、Memory/Knowledge、Reasoning/Evaluation、Planning/Feedback
5. **知识地图**：全书13大主题，覆盖基础入门、核心组件、实战应用三个部分

第2章：访问大语言模型

2.1 LLM访问方式对比

在构建AI Agent之前，首先需要了解如何访问大语言模型。本节对比主流的LLM访问方式。

2.1.1 主流LLM提供商概览

提供商	代表模型	API类型	优势	适用场景
OpenAI	GPT-4o, GPT-4 Turbo, GPT-3.5 Turbo	REST, WebSocket	性能最强生态成熟	企业级应用、通用场景
Azure OpenAI	GPT-4, GPT-3.5	REST	企业安全合规、SLA保障	大型企业、敏感数据
Anthropic	Claude 3.5 Sonnet, Claude 3 Opus	REST, Messages	长上下文、安全性强	长文档处理、内容创作
Google	Gemini 1.5 Pro, Gemini 1.0 Pro	REST, SDK	多模态、长上下文	多模态任务、长文本
Groq	LLaMA 3, Mixtral	REST, WebSocket	极速推理、低延迟	实时交互场景
LM Studio	本地模型	REST (Local)	完全私有、成本低	数据隐私敏感场景
Ollama	LLaMA 3, Mistral, Qwen	REST (Local)	易于部署、轻量	本地开发、原型验证

图2-1：LLM提供商访问方式对比表

```
┌──────────────
```

```python
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "你是一个有用的助手。"},
        {"role": "user", "content": "解释什么是AI Agent"}
    ],
    temperature=0.7,
    max_tokens=1000
)

print(response.choices[0].message.content)
```

特点:

- 模型选择丰富（GPT-4o, GPT-4 Turbo, GPT-3.5 Turbo）
- 生态最完善，集成工具多
- 全球CDN加速，延迟低
- 按token计费，成本可控

2.1.3 Azure OpenAI

Azure OpenAI提供企业级的LLM访问:

```python
from openai import AzureOpenAI

client = AzureOpenAI(
    api_key="your-api-key",
    api_version="2024-02-01",
    azure_endpoint="https://your-resource.openai.azure.com/"
)

response = client.chat.completions.create(
    model="gpt-4o",  # deployment name
    messages=[
        {"role": "system", "content": "你是一个有用的助手。"},
        {"role": "user", "content": "解释什么是AI Agent"}
    ]
)
```

优势:

- Azure AD认证，企业级安全
- 虚拟网络隔离
- 数据不用于模型训练（承诺）
- SLA服务保障

2.1.4 Anthropic Claude

Claude以其长上下文和安全性著称:

```python
from anthropic import Anthropic

client = Anthropic(api_key="your-api-key")

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1024,
    messages=[
        {"role": "user", "content": "解释什么是AI Agent"}
    ]
)

print(message.content[0].text)
```

优势:

- 200K token超长上下文
- 强大的代码能力

- 内置安全对齐
- 擅长长文档分析

2.1.5 Google Gemini

Google Gemini是多模态能力的代表：

```
import google.generativeai as genai

genai.configure(api_key="your-api-key")

model = genai.GenerativeModel('gemini-1.5-pro')

response = model.generate_content("解释什么是AI Agent")
print(response.text)
```

优势：

- 原生多模态（文本/图片/视频/音频）
- 100万token上下文（Gemini 1.5）
- Google生态集成
- 高性价比

2.1.6 Groq

Groq以极速推理著称：

```
from groq import Groq

client = Groq(api_key="your-api-key")

response = client.chat.completions.create(
    model="llama-3.1-70b-versatile",
    messages=[
        {"role": "user", "content": "解释什么是AI Agent"}
    ],
    temperature=0.5,
    max_tokens=1024
)

print(response.choices[0].message.content)
```

优势：

- LPU推理引擎，极低延迟
- 支持开源模型
- 按token计费
- 实时交互首选

2.1.7 LM Studio

LM Studio支持本地模型部署：

```
from openai import OpenAI

# 连接本地LM Studio服务器
client = OpenAI(
    base_url="http://localhost:1234/v1",
    api_key="not-needed"
)

response = client.chat.completions.create(
    model="local-model",
    messages=[
        {"role": "user", "content": "解释什么是AI Agent"}
    ]
)

print(response.choices[0].message.content)
```

2.1.8 Ollama

Ollama是轻量级本地部署方案：

```
# 安装模型
ollama pull llama3.1

# 运行模型
ollama run llama3.1 "解释什么是AI Agent"
```

```
import ollama

response = ollama.chat(
    model='llama3.1',
    messages=[
        {"role": "user", "content": "解释什么是AI Agent"}
    ]
)

print(response['message']['content'])
```

2.2 核心API参数详解

2.2.1 参数一览表

参数	类型	说明	常见取值
model	string	使用的模型名称	gpt-4o, claude-3-5-sonnet-20241022, gemini-1.5-pro
messages	array	对话消息列表	[{role, content}, ...]
temperature	float	随机性控制	0.0-2.0（默认0.7）
top_p	float	核采样阈值	0.0-1.0（默认1.0）
max_tokens	int	最大生成token数	100-32000
system_prompt	string	系统提示词	角色定义/行为规则

2.2.2 messages参数详解

messages是对话的核心数据结构：

```
messages = [
    # 系统提示：定义Agent身份和行为
    {
        "role": "system",
        "content": """你是"小智"，一个专业的Python编程助手。
        - 擅长Python代码编写和调试
        - 解释技术概念简洁清晰
        - 主动提供最佳实践建议"""
    },

    # 历史消息：维持对话上下文
    {
        "role": "user",
        "content": "什么是装饰器？"
    },
    {
        "role": "assistant",
        "content": "装饰器是Python中的一种高级语法..."
    },

    # 当前消息：最新用户输入
    {
        "role": "user",
        "content": "给我一个实际例子"
    }
]
```

role类型：

role	说明	示例

system	系统提示，定义行为	角色设定、规则定义
user	用户消息	人类输入、指令
assistant	助手回复	AI生成的响应

2.2.3 temperature参数详解

temperature控制输出的**随机性**：

```
temperature = 0.0（确定性强）
    ↓
┌──────────────────────────────────────────────────────────────┐
│ 几乎总是选择最高概率的token                                    │
│ 适合：精确问答、代码生成、翻译                                  │
└──────────────────────────────────────────────────────────────┘

temperature = 0.7（平衡）
    ↓
┌──────────────────────────────────────────────────────────────┐
│ 在高概率和低概率之间取得平衡                                    │
│ 适合：一般对话、内容创作                                        │
└──────────────────────────────────────────────────────────────┘

temperature = 1.5（高随机性）
    ↓
┌──────────────────────────────────────────────────────────────┐
│ 更多样化的输出，可能更有创意                                    │
│ 适合：头脑风暴、创意写作                                        │
└──────────────────────────────────────────────────────────────┘

temperature = 2.0（极高随机性）
    ↓
┌──────────────────────────────────────────────────────────────┐
│ 输出可能变得不连贯、无意义                                      │
│ 通常不推荐使用                                                 │
└──────────────────────────────────────────────────────────────┘
```

场景推荐：

场景	建议temperature
代码生成	0.0 - 0.3
精确问答	0.0 - 0.2
事实性解释	0.0 - 0.3
一般对话	0.7 - 0.8
创意写作	0.8 - 1.0
头脑风暴	1.0 - 1.2

2.2.4 top_p参数详解

top_p是另一种控制随机性的方式（核采样）：

```
top_p = 1.0（不使用核采样）
    ↓
考虑所有token，概率分布完整

top_p = 0.9（常用）
    ↓
┌──────────────────────────────────────────────────────────────┐
│ 只考虑累计概率达到90%的token                                   │
│ 跳过低概率token，减少噪声                                      │
│ 效果类似中等temperature                                        │
└──────────────────────────────────────────────────────────────┘

top_p = 0.5
    ↓
只考虑最核心的高概率token，更确定性
```

```
top_p = 0.1
    ↓
几乎只选择最高概率的少数token
```

使用建议：

- 通常**只设置temperature或top_p之一**，不要同时设置
- `temperature=0.0, top_p=1.0` = 完全确定性
- `temperature=1.0, top_p=0.95` = 保留多样性但避免极端

2.2.5 max_tokens参数详解

max_tokens限制单次生成的最大token数：

```
# 限制输出长度
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "写一首诗"}],
    max_tokens=100  # 限制在约100个token
)
```

计算参考：

max_tokens	约等于中文	约等于英文
100	50-80字	75-100词
500	250-400字	375-500词
1000	500-800字	750-1000词
2000	1000-1600字	1500-2000词

注意事项：

- max_tokens是**最大限制**，不是固定输出
- 输入+输出的总token数不能超过模型的上下文窗口
- 设置过小会导致输出被截断

2.2.6 system_prompt设计原则

system_prompt是定义Agent行为的核心：

```
# 好的system_prompt结构
system_prompt = """
# 角色定义
你是一个[专业领域]助手。

# 核心能力
- 擅长...
- 能够...

# 行为规范
- 始终...
- 避免...

# 输出格式
回答时使用...
"""
```

最佳实践：

1. **清晰定义角色**：明确Agent是什么、能做什么
2. **具体能力边界**：说明擅长什么、不擅长什么
3. **指定输出格式**：如JSON、Markdown、纯文本
4. **举例说明**：用few-shot示例展示期望行为

2.3 API调用成本分析与优化

2.3.1 主流模型价格对比

模型	输入价格（$/1M tokens）	输出价格（$/1M tokens）	上下文窗口

GPT-4o	$2.50	$10.00	128K
GPT-4o Mini	$0.15	$0.60	128K
GPT-4 Turbo	$10.00	$30.00	128K
Claude 3.5 Sonnet	$3.00	$15.00	200K
Claude 3 Opus	$15.00	$75.00	200K
Gemini 1.5 Pro	$1.25	$5.00	1M
Gemini 1.5 Flash	$0.075	$0.30	1M
LLaMA 3.1 70B	$0.65	$2.75	128K
LLaMA 3.1 8B	$0.15	$0.40	128K

2.3.2 成本优化策略

策略一：选择合适的模型

```
# 复杂任务 → 使用强模型
response = client.chat.completions.create(
    model="gpt-4o",  # 高成本但高质量
    messages=[{"role": "user", "content": "分析这份法律合同的潜在风险"}]
)

# 简单任务 → 使用轻量模型
response = client.chat.completions.create(
    model="gpt-4o-mini",  # 低成本足以处理
    messages=[{"role": "user", "content": "今天天气怎么样？"}]
)
```

策略二：减少token消耗

```
# ❌ 低效：冗长的system prompt
system_prompt = """
你是一个非常非常有用的助手。
你具备丰富的知识和专业技能。
你能够帮助你解决各种问题。
你总是努力提供最准确、最有用的信息。
"""

# ✅ 高效：简洁的system prompt
system_prompt = "你是一个有用的助手。"
```

策略三：批量处理

```
# ❌ 低效：多次单独调用
for query in queries:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": query}]
    )

# ✅ 高效：批量处理
batch_prompt = "\n".join([f"Q{i+1}: {q}" for i, q in enumerate(queries)])
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{
        "role": "user",
        "content": f"回答以下问题：\n{batch_prompt}"
    }]
)
```

策略四：缓存重复内容

```
# 使用缓存减少重复token计费
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "解释量子计算"}],
```

```
    extra_headers={"X-Client-Tracing-Id": "session-123"}  # 启用上下文缓存
)
```

2.3.3 成本监控示例

```python
import tiktoken  # OpenAI的token计算库

def estimate_cost(prompt: str, model: str = "gpt-4o-mini") -> dict:
    """估算API调用成本"""
    enc = tiktoken.encoding_for_model(model)

    # 计算token数
    input_tokens = len(enc.encode(prompt))

    # 价格（$/1M tokens）
    prices = {
        "gpt-4o": (2.5, 10.0),
        "gpt-4o-mini": (0.15, 0.60),
        "claude-3-5-sonnet": (3.0, 15.0)
    }

    input_price, output_price = prices.get(model, (0, 0))

    # 估算输出成本（假设输出是输入的30%）
    estimated_output = int(input_tokens * 0.3)

    total_cost = (
        input_tokens * input_price / 1_000_000 +
        estimated_output * output_price / 1_000_000
    )

    return {
        "input_tokens": input_tokens,
        "estimated_output_tokens": estimated_output,
        "estimated_cost_usd": round(total_cost, 6)
    }

# 使用示例
result = estimate_cost("解释什么是AI Agent，包括其核心组件和典型应用场景")
print(f"输入Token: {result['input_tokens']}")
print(f"预估输出Token: {result['estimated_output_tokens']}")
print(f"预估成本: ${result['estimated_cost_usd']}")
```

2.4 本地模型部署：LM Studio与Ollama

图2-3：本地模型部署架构图

```
┌──────────────
```

```
│   │   │   (LM Studio)      │     │     (Ollama)        │               │    │
│   │   └──────────────────────────────────────┘     └──────────────────────────────────────┘               │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                           │                   │                   │
│                           ↓                   ↓                   │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │                     模型服务层                                  │    │
│   │   ┌──────────────────────────────────────┐     ┌──────────────────────────────────────┐               │    │
│   │   │   LM Studio        │     │    Ollama         │               │    │
│   │   │   ┌──────────────────────────┐   │     │   ┌──────────────────────────┐   │               │    │
│   │   │   │ GGUF/GGML    │   │     │   │   Ollama模型 │   │               │    │
│   │   │   │ 模型格式     │   │     │   │  (Container) │   │               │    │
│   │   │   └──────────────────────────┘   │     │   └──────────────────────────┘   │               │    │
│   │   │   ┌──────────────────────────┐   │     │   ┌──────────────────────────┐   │               │    │
│   │   │   │ CUDA/Metal   │   │     │   │   Llama.cpp  │   │               │    │
│   │   │   │ 推理加速     │   │     │   │   推理后端   │   │               │    │
│   │   │   └──────────────────────────┘   │     │   └──────────────────────────┘   │               │    │
│   │   └──────────────────────────────────────┘     └──────────────────────────────────────┘               │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                           │                   │                   │
│                           ↓                   ↓                   │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │                     硬件层                                      │    │
│   │   ┌──────────────────────────┐   ┌──────────────────────────┐   ┌──────────────────────────┐         │    │
│   │   │    GPU       │   │     CPU      │   │     RAM      │         │    │
│   │   │  (NVIDIA)    │   │   Apple M    │   │   16GB+      │         │    │
│   │   │  RTX 3090+   │   │    Silicon   │   │              │         │    │
│   │   └──────────────────────────┘   └──────────────────────────┘   └──────────────────────────┘         │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                   │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

2.4.1 LM Studio安装与配置

下载地址：https://lmstudio.ai/

安装步骤：

1. 下载对应操作系统的安装包
2. 运行安装程序
3. 启动LM Studio桌面应用

下载模型：

1. 在搜索栏中搜索模型（如 "LLaMA 3.1"）
2. 选择模型大小（7B、8B、70B）
3. 点击下载
4. 等待下载完成

启动本地服务器：

1. 点击左侧"Local Server"图标
2. 选择已下载的模型
3. 设置端口（默认1234）
4. 点击"Start Server"

API调用示例：

```
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:1234/v1",
    api_key="not-needed"  # 本地服务不需要API Key
)

response = client.chat.completions.create(
    model="llama-3.1-8b-instruct",  # 模型名称
    messages=[
```

```
        {"role": "system", "content": "你是一个有用的助手。"},
        {"role": "user", "content": "你好！"}
    ],
    temperature=0.7,
    max_tokens=500
)

print(response.choices[0].message.content)
```

2.4.2 Ollama安装与配置

安装:

```
# macOS/Linux
curl -fsSL https://ollama.com/install.sh | sh

# Windows（通过Docker或WSL）
# 参考：https://github.com/ollama/ollama
```

常用命令:

```
# 查看可用模型
ollama list

# 下载模型
ollama pull llama3.1        # 下载LLaMA 3.1
ollama pull mistral         # 下载Mistral
ollama pull qwen2.5         # 下载通义千问

# 运行模型
ollama run llama3.1 "你好"

# 创建自定义模型
ollama create my-agent -f Modelfile

# 停止运行中的模型
ollama stop llama3.1
```

Modelfile自定义配置:

```
# Modelfile
FROM llama3.1

# 设置系统提示
SYSTEM """
你是一个专业的Python编程助手。
- 擅长代码编写和调试
- 解释技术概念清晰易懂
"""

# 设置参数
PARAMETER temperature 0.7
PARAMETER top_p 0.9
PARAMETER num_ctx 4096
```

```
# 创建自定义模型
ollama create python-assistant -f Modelfile

# 运行自定义模型
ollama run python-assistant
```

Ollama API调用:

```
import ollama

# 同步调用
response = ollama.chat(
    model='llama3.1',
```

```
    messages=[
        {"role": "user", "content": "解释什么是生成式AI"}
    ],
    options={
        "temperature": 0.7,
        "num_predict": 500
    }
)

print(response['message']['content'])

# 流式输出
stream = ollama.chat(
    model='llama3.1',
    messages=[
        {"role": "user", "content": "写一个Python快速排序"}
    ],
    stream=True
)

for chunk in stream:
    print(chunk['message']['content'], end='', flush=True)
```

2.4.3 LM Studio与Ollama对比

特性	LM Studio	Ollama
易用性	图形界面，操作直观	命令行为主
模型格式	GGUF/GGML	Ollama专用格式
模型市场	内置模型下载	社区模型库
API兼容性	OpenAI兼容	独立API
GPU加速	自动检测	自动配置
适用场景	桌面应用、临时测试	服务器、长期部署
资源占用	较高	较低
自定义程度	中等	高

2.4.4 性能对比参考

模型	硬件要求	推理速度（tokens/s）	适用场景
LLaMA 3.1 8B	8GB RAM, 无GPU	15-25	原型开发、轻量任务
LLaMA 3.1 70B	64GB RAM, 24GB GPU	30-50	复杂推理、代码生成
Mistral 7B	16GB RAM, 8GB GPU	40-60	平衡性能与资源
Qwen 2.5 72B	128GB RAM, 48GB GPU	20-40	中文任务、专业领域

2.5 系统Prompt工程：如何定义Agent行为

图2-4：系统Prompt设计模板图

```
┌──────────────
```

```
│                                                          │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│
│  │  #2 核心能力（Capabilities）                                  │   │
│  │  ──────────────────────────────────────────────────────────────────────────────────────────  │   │
│  │                                                          │   │
│  │  ## 你能做什么                                              │   │
│  │  - 精通[技能1]、[技能2]、[技能3]                                 │   │
│  │  - 能够[能力描述]                                            │   │
│  │  - 擅长处理[任务类型]                                         │   │
│  │                                                          │   │
│  │  ## 你不能做什么                                             │   │
│  │  - 不擅长[限制领域]                                           │   │
│  │  - 不会[限制行为]                                             │   │
│  │                                                          │   │
│  └──────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                          │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│
│  │  #3 行为规范（Behavior Guidelines）                           │   │
│  │  ──────────────────────────────────────────────────────────────────────────────────────────  │   │
│  │                                                          │   │
│  │  ## 沟通风格                                                │   │
│  │  - 表达方式：[正式/友好/专业/简洁]                                 │   │
│  │  - 使用语言：[中文/英文/双语]                                     │   │
│  │  - 回答长度：[简短/中等/详细]                                     │   │
│  │                                                          │   │
│  │  ## 交互原则                                                │   │
│  │  - 遇到不确定的问题：[直接承认/猜测后说明]                          │   │
│  │  - 需要更多信息时：[主动询问/基于现有信息]                          │   │
│  │  - 发现错误时：[主动纠正/等待用户指出]                              │   │
│  │                                                          │   │
│  └──────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                          │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│
│  │  #4 输出格式（Output Format）                                 │   │
│  │  ──────────────────────────────────────────────────────────────────────────────────────────  │   │
│  │                                                          │   │
│  │  ## 结构化输出                                              │   │
│  │  当需要输出代码时：                                            │   │
│  │  ```[语言]                                                 │   │
│  │  代码                                                       │   │
│  │  ```                                                       │   │
│  │                                                          │   │
│  │  当需要分析时，使用以下格式：                                      │   │
│  │  - 优点：...                                                 │   │
│  │  - 缺点：...                                                 │   │
│  │  - 建议：...                                                 │   │
│  │                                                          │   │
│  └──────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                          │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│
│  │  #5 示例（Few-Shot Examples）                                 │   │
│  │  ──────────────────────────────────────────────────────────────────────────────────────────  │   │
│  │                                                          │   │
│  │  示例1：                                                     │   │
│  │  用户：xxx                                                   │   │
│  │  助手：yyy                                                   │   │
│  │                                                          │   │
│  │  示例2：                                                     │   │
│  │  用户：aaa                                                   │   │
│  │  助手：bbb                                                   │   │
│  │                                                          │   │
│  └──────────────────────────────────────────────────────────────────────────────────────────────┘
│
```

```
│                                                                │
└────────────
```

```
        self.max_tokens = max_tokens
        self.messages = []
        self.total_tokens = 0

    def add_message(self, role: str, content: str):
        """添加消息并估算token"""
        token_count = len(content) // 4  # 粗略估算
        self.messages.append({"role": role, "content": content})
        self.total_tokens += token_count

        # 如果超限，压缩早期消息
        while self.total_tokens > self.max_tokens and len(self.messages) > 2:
            removed = self.messages.pop(1)  # 保留第一条system消息
            self.total_tokens -= len(removed["content"]) // 4

    def get_messages(self):
        return self.messages

# 使用示例
manager = ConversationManager(max_tokens=4000)
manager.add_message("system", "你是一个有用的助手。")
manager.add_message("user", "你好")
manager.add_message("assistant", "你好！有什么可以帮助你的吗？")
manager.add_message("user", "帮我解释什么是机器学习")
# ... 继续对话
```

2.6 多模型统一接口设计

2.6.1 抽象层设计

```
from abc import ABC, abstractmethod
from typing import List, Dict, Optional, Any
from enum import Enum

class ModelProvider(Enum):
    OPENAI = "openai"
    ANTHROPIC = "anthropic"
    GOOGLE = "google"
    OLLAMA = "ollama"
    LMSTUDIO = "lmstudio"

class LLMInterface(ABC):
    """LLM统一接口抽象类"""

    @abstractmethod
    def chat(self, messages: List[Dict], **kwargs) -> str:
        """发送对话请求，返回回复内容"""
        pass

    @abstractmethod
    def chat_stream(self, messages: List[Dict], **kwargs):
        """流式对话"""
        pass

    @abstractmethod
    def get_provider(self) -> ModelProvider:
        """获取提供商标识"""
        pass
```

2.6.2 OpenAI实现

```
from openai import OpenAI

class OpenAIClient(LLMInterface):
    def __init__(self, api_key: str, model: str = "gpt-4o-mini",
                 base_url: Optional[str] = None):
        self.client = OpenAI(
            api_key=api_key,
```

```
            base_url=base_url  # 用于兼容本地部署
        )
        self.model = model

    def chat(self, messages: List[Dict], **kwargs) -> str:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=messages,
            temperature=kwargs.get("temperature", 0.7),
            max_tokens=kwargs.get("max_tokens", 1000)
        )
        return response.choices[0].message.content

    def chat_stream(self, messages: List[Dict], **kwargs):
        stream = self.client.chat.completions.create(
            model=self.model,
            messages=messages,
            stream=True,
            temperature=kwargs.get("temperature", 0.7)
        )
        for chunk in stream:
            if chunk.choices[0].delta.content:
                yield chunk.choices[0].delta.content

    def get_provider(self) -> ModelProvider:
        return ModelProvider.OPENAI
```

2.6.3 Anthropic实现

```
from anthropic import Anthropic

class AnthropicClient(LLMInterface):
    def __init__(self, api_key: str, model: str = "claude-3-5-sonnet-20241022"):
        self.client = Anthropic(api_key=api_key)
        self.model = model

    def chat(self, messages: List[Dict], **kwargs) -> str:
        # 分离system消息
        system_prompt = ""
        filtered_messages = []

        for msg in messages:
            if msg["role"] == "system":
                system_prompt = msg["content"]
            else:
                filtered_messages.append(msg)

        response = self.client.messages.create(
            model=self.model,
            system=system_prompt,
            messages=filtered_messages,
            max_tokens=kwargs.get("max_tokens", 1024),
            temperature=kwargs.get("temperature", 0.7)
        )

        return response.content[0].text

    def chat_stream(self, messages: List[Dict], **kwargs):
        # Anthropic流式实现
        pass

    def get_provider(self) -> ModelProvider:
        return ModelProvider.ANTHROPIC
```

2.6.4 Ollama实现

```
import ollama
```

```python
class OllamaClient(LLMInterface):
    def __init__(self, model: str = "llama3.1", base_url: str = "http://localhost:11434"):
        self.model = model
        self.base_url = base_url

    def chat(self, messages: List[Dict], **kwargs) -> str:
        response = ollama.chat(
            model=self.model,
            messages=messages,
            options={
                "temperature": kwargs.get("temperature", 0.7),
                "num_predict": kwargs.get("max_tokens", 1000)
            }
        )
        return response['message']['content']

    def chat_stream(self, messages: List[Dict], **kwargs):
        stream = ollama.chat(
            model=self.model,
            messages=messages,
            stream=True
        )
        for chunk in stream:
            yield chunk['message']['content']

    def get_provider(self) -> ModelProvider:
        return ModelProvider.OLLAMA
```

2.6.5 统一调度器

```python
class LLMDispatcher:
    """多模型统一调度器"""

    def __init__(self):
        self.clients: Dict[ModelProvider, LLMInterface] = {}
        self.default_provider = ModelProvider.OPENAI

    def register(self, client: LLMInterface):
        """注册LLM客户端"""
        self.clients[client.get_provider()] = client

    def chat(self, messages: List[Dict],
             provider: Optional[ModelProvider] = None,
             **kwargs) -> str:
        """统一对话接口"""
        provider = provider or self.default_provider

        if provider not in self.clients:
            raise ValueError(f"Provider {provider} not registered")

        return self.clients[provider].chat(messages, **kwargs)

    def chat_with_fallback(self, messages: List[Dict],
                           providers: List[ModelProvider] = None,
                           **kwargs) -> str:
        """级联调用，失败时自动切换"""
        providers = providers or [self.default_provider]

        errors = []
        for provider in providers:
            try:
                return self.chat(messages, provider, **kwargs)
            except Exception as e:
                errors.append(f"{provider.value}: {str(e)}")

        raise Exception(f"All providers failed: {errors}")

# 使用示例
dispatcher = LLMDispatcher()
```

```
dispatcher.register(OpenAIClient(api_key="sk-xxx", model="gpt-4o"))
dispatcher.register(AnthropicClient(api_key="sk-ant-xxx"))
dispatcher.register(OllamaClient(model="llama3.1"))

# 统一调用
response = dispatcher.chat(
    messages=[{"role": "user", "content": "你好"}],
    provider=ModelProvider.OPENAI  # 指定提供商
)

# 自动降级
response = dispatcher.chat_with_fallback(
    messages=[{"role": "user", "content": "你好"}],
    providers=[ModelProvider.OPENAI, ModelProvider.ANTHROPIC, ModelProvider.OLLAMA]
)
```

2.6.6 成本与性能路由器

```
class SmartRouter:
    """智能路由：根据任务自动选择最合适的模型"""

    # 任务复杂度定义
    TASK_COMPLEXITY = {
        "simple_qa": {"providers": ["openai-gpt4o-mini", "ollama"], "max_cost": 0.001},
        "code_generation": {"providers": ["openai-gpt4o", "anthropic"], "max_cost": 0.01},
        "long_analysis": {"providers": ["anthropic", "google"], "max_cost": 0.05},
        "creative": {"providers": ["openai-gpt4o", "ollama"], "max_cost": 0.02}
    }

    def __init__(self, dispatcher: LLMDispatcher):
        self.dispatcher = dispatcher

    def route(self, task_type: str, messages: List[Dict], **kwargs) -> str:
        """根据任务类型路由"""
        if task_type not in self.TASK_COMPLEXITY:
            task_type = "simple_qa"

        config = self.TASK_COMPLEXITY[task_type]
        providers = config["providers"]

        # 尝试每个提供商直到成功
        for provider_key in providers:
            try:
                provider = ModelProvider(provider_key.split("-")[0])
                return self.dispatcher.chat(messages, provider, **kwargs)
            except Exception as e:
                print(f"Provider {provider_key} failed: {e}")
                continue

        raise Exception("All providers exhausted")

    def route_by_content(self, messages: List[Dict]) -> str:
        """根据内容复杂度自动路由"""
        content = messages[-1]["content"]

        # 简单判断：长度+关键词
        if len(content) < 50 and "解释" not in content:
            task_type = "simple_qa"
        elif any(kw in content for kw in ["代码", "程序", "函数", "class"]):
            task_type = "code_generation"
        elif len(content) > 2000:
            task_type = "long_analysis"
        else:
            task_type = "simple_qa"

        return self.route(task_type, messages)

# 使用示例
router = SmartRouter(dispatcher)
```

```
response = router.route("code_generation", messages)
response = router.route_by_content(messages)
```

本章小结

本章介绍了访问大语言模型的各种方式：

1. **提供商对比**：OpenAI、Azure OpenAI、Anthropic、Google、Groq、LM Studio、Ollama各有特点
2. **核心API参数**：model、messages、temperature、top_p、max_tokens是控制输出质量和成本的关键
3. **成本优化**：选择合适模型、减少token消耗、批量处理、启用缓存
4. **本地部署**：LM Studio适合桌面应用，Ollama适合服务器部署
5. **Prompt工程**：清晰具体的指令、结构化输出、few-shot示例
6. **统一接口设计**：通过抽象层和调度器实现多模型统一调用

第3章：构建你的第一个Agent

3.1 GPT Assistants简介

3.1.1 什么是GPT Assistants

GPT Assistants是OpenAI提供的Agent构建框架，允许开发者创建具有持久线程、文件访问和自定义操作的AI助手。

核心特性：

特性	说明	优势
持久线程（Threads）	自动管理对话历史	无需手动维护上下文
文件访问（Files）	支持上传和检索文件	可处理PDF、文档、代码
自定义工具（Tools）	定义Function Calling	扩展Agent执行能力
指令控制（Instructions）	设置助手的系统提示	精确控制行为
模型选择（Model）	可选择GPT-4o等模型	平衡性能与成本

图3-1：GPT Assistants架构图

```
┌──────────────
```

```
│   └──────────────────────────────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                  │                                                  │
│                                                  ↓                                                  │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │                                    Run 执行流程                                    │    │
│   │                                                                                      │    │
│   │    ┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐       │    │
│   │    │ Thread  │──────▶│  Run    │──────▶│ Steps   │──────▶│ Messages│       │    │
│   │    │ (对话)  │     │ (执行)  │     │ (步骤)  │     │ (结果)  │       │    │
│   │    └──────────────┘     └──────────┬───────┘     └──────────────┘     └──────────────┘       │    │
│   │                    │                                                    │    │
│   │                    ↓                                                    │    │
│   │              ┌──────────────────────┐                                          │    │
│   │              │   Tools     │                                    │    │
│   │              │  ┌──────────────┐  │                                          │    │
│   │              │  │ Code    │ │                                    │    │
│   │              │  │ Interpret│ │                                    │    │
│   │              │  └──────────────┘  │                                          │    │
│   │              │  ┌──────────────┐  │                                          │    │
│   │              │  │ File    │ │                                    │    │
│   │              │  │ Search  │ │                                    │    │
│   │              │  └──────────────┘  │                                          │    │
│   │              │  ┌──────────────┐  │                                          │    │
│   │              │  │Custom Fn│ │                                    │    │
│   │              │  └──────────────┘  │                                          │    │
│   │              └──────────────────────┘                                          │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

3.1.2 Assistants vs 普通API

对比维度	普通ChatGPT API	GPT Assistants
上下文管理	手动维护messages	自动管理线程
文件处理	需手动编码解码	内置文件上传检索
代码执行	需搭建沙箱环境	内置代码解释器
工具调用	手动实现Function Calling	原生支持多种工具
状态保持	无状态	持久化助手配置
适用场景	简单问答	复杂任务助手
开发复杂度	低	中等
成本	按token计费	按token+工具计费

3.2 OpenAI Assistants Playground使用指南

3.2.1 Playground界面介绍

访问地址：https://platform.openai.com/playground

主要区域：

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│                    OpenAI Assistants Playground                          │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │  [Assistant ▼] [Model: gpt-4o    ▼] [Temperature: 0.7] [Tools] │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                  │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │  INSTRUCTIONS                                          │    │
```

```
│   │  ──────────────────────────────────────────────────────────────────────────────────────────────────────────────  │   │
│   │   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────┐  │   │
│   │   │ 你是一个Python编程助手。                                                                                  │  │   │
│   │   │ • 擅长代码编写和调试                                                                                      │  │   │
│   │   │ • 解释技术概念清晰易懂                                                                                    │  │   │
│   │   │ • 提供最佳实践建议                                                                                        │  │   │
│   │   └──────────────────────────────────────────────────────────────────────────────────────────────────────────┘  │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                  │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │  TOOLS                                                                                                          │   │
│   │  ──────────────────────────────────────────────────────────────────────────────────────────────────────────────  │   │
│   │  ☑ Code Interpreter                                                                                             │   │
│   │  ☑ File Search                                                                                                  │   │
│   │  ☑ Functions                                                                                                    │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                  │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │  FILES                                                                                                          │   │
│   │  ──────────────────────────────────────────────────────────────────────────────────────────────────────────────  │   │
│   │  + Add files                                                                                                    │   │
│   │  [document.pdf] [data.csv] [code.py]                                                                            │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                  │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │                                CHAT                                                                             │   │
│   │  ──────────────────────────────────────────────────────────────────────────────────────────────────────────────  │   │
│   │                                                                                                                 │   │
│   │  [User] 你能帮我写一个快速排序算法吗？                                                                            │   │
│   │                                                                                                                 │   │
│   │  [Assistant] 当然可以！以下是一个Python实现的快速排序算法：                                                        │   │
│   │                                                                                                                 │   │
│   │  ```python                                                                                                      │   │
│   │  def quicksort(arr):                                                                                            │   │
│   │      if len(arr) <= 1:                                                                                          │   │
│   │          return arr                                                                                             │   │
│   │      ...                                                                                                        │   │
│   │  ```                                                                                                            │   │
│   │                                                                                                                 │   │
│   │  [User] 能解释一下时间复杂度吗？                                                                                  │   │
│   │                                                                                                                 │   │
│   │  [Assistant] 快速排序的平均时间复杂度是O(n log n)...                                                              │   │
│   │                                                                                                                 │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                  │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │  [Create Assistant]                    [Submit]                                                                 │   │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

3.2.2 快速创建助手步骤

Step 1：设置Instructions

在Instructions框中输入助手的系统提示，定义其角色和能力。

Step 2：启用工具

根据需求启用相关工具：

- **Code Interpreter**：代码执行环境
- **File Search**：向量知识检索

- **Functions**：自定义API调用

Step 3：上传文件（可选）

点击"Add files"上传参考资料，如：

- PDF文档（用于问答）
- CSV数据（用于分析）
- 代码文件（用于调试）

Step 4：测试对话

在Chat区域输入问题测试助手效果。

Step 5：创建Assistant

点击"Create Assistant"生成API代码。

3.3 工具定义：Function Calling机制与参数自动解析

图3-2：Function Calling工作流程图

```
┌──────────────
```

```
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                   ↓                                                  │
│   Step 4: 结果整合                                                                   │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │   将函数结果返回给模型                                                                  │    │
│   │   model.generate(                                                                     │    │
│   │     messages + [                                                                      │    │
│   │       {"role": "assistant", "function_call": ...},                                    │    │
│   │       {"role": "function", "content": json.dumps(result)}                             │    │
│   │     ]                                                                                 │    │
│   │   )                                                                                   │    │
│   │                                   ↓                                                   │    │
│   │   最终回复："北京今天天气晴朗，气温22°C，湿度45%"                                          │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                                          │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

3.3.1 函数定义规范

Function Calling使用JSON Schema定义函数签名：

```python
# 函数定义示例
functions = [
    {
        # 函数名称
        "name": "get_weather",

        # 函数功能描述（帮助模型判断何时调用）
        "description": "获取指定城市的当前天气信息，包括温度、天气状况和湿度",

        # 参数规范（JSON Schema格式）
        "parameters": {
            "type": "object",
            "properties": {
                "city": {
                    "type": "string",
                    "description": "城市名称，使用中文，如'北京'、'上海'"
                },
                "unit": {
                    "type": "string",
                    "enum": ["celsius", "fahrenheit"],
                    "description": "温度单位，默认celsius",
                    "default": "celsius"
                }
            },
            "required": ["city"]
        }
    },
    {
        "name": "send_email",
        "description": "发送电子邮件",
        "parameters": {
            "type": "object",
            "properties": {
                "to": {
                    "type": "string",
                    "description": "收件人邮箱地址"
                },
                "subject": {
                    "type": "string",
                    "description": "邮件主题"
                },
                "body": {
                    "type": "string",
                    "description": "邮件正文内容"
                }
            },
```

```
            "required": ["to", "subject", "body"]
        }
    },
    {
        "name": "create_reminder",
        "description": "创建日程提醒",
        "parameters": {
            "type": "object",
            "properties": {
                "title": {
                    "type": "string",
                    "description": "提醒标题"
                },
                "datetime": {
                    "type": "string",
                    "description": "提醒时间，格式为YYYY-MM-DD HH:MM"
                },
                "description": {
                    "type": "string",
                    "description": "提醒描述（可选）"
                }
            },
            "required": ["title", "datetime"]
        }
    }
]
```

3.3.2 函数调用处理

```python
from openai import OpenAI
import json

client = OpenAI(api_key="your-api-key")

def get_weather(city: str, unit: str = "celsius") -> dict:
    """模拟获取天气信息"""
    # 实际项目中这里会调用天气API
    return {
        "city": city,
        "temperature": 22,
        "condition": "晴朗",
        "humidity": 45,
        "unit": unit
    }

def send_email(to: str, subject: str, body: str) -> dict:
    """模拟发送邮件"""
    # 实际项目中这里会调用邮件服务API
    return {
        "status": "sent",
        "to": to,
        "subject": subject,
        "timestamp": "2024-01-15T10:30:00Z"
    }

def create_reminder(title: str, datetime: str, description: str = None) -> dict:
    """模拟创建提醒"""
    # 实际项目中这里会调用日历API
    return {
        "status": "created",
        "title": title,
        "datetime": datetime,
        "id": "reminder_12345"
    }

# 函数映射表
function_map = {
    "get_weather": get_weather,
    "send_email": send_email,
```

```python
    "create_reminder": create_reminder
}

# 对话函数
def chat_with_functions(user_message: str):
    messages = [
        {"role": "system", "content": "你是一个助手，可以使用函数来完成任务。"},
        {"role": "user", "content": user_message}
    ]

    # 第一次调用
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=[
            {
                "type": "function",
                "function": {
                    "name": "get_weather",
                    "description": "获取指定城市的天气信息",
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "city": {"type": "string", "description": "城市名称"},
                            "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
                        },
                        "required": ["city"]
                    }
                }
            },
            {
                "type": "function",
                "function": {
                    "name": "send_email",
                    "description": "发送电子邮件",
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "to": {"type": "string"},
                            "subject": {"type": "string"},
                            "body": {"type": "string"}
                        },
                        "required": ["to", "subject", "body"]
                    }
                }
            },
            {
                "type": "function",
                "function": {
                    "name": "create_reminder",
                    "description": "创建日程提醒",
                    "parameters": {
                        "type": "object",
                        "properties": {
                            "title": {"type": "string"},
                            "datetime": {"type": "string"},
                            "description": {"type": "string"}
                        },
                        "required": ["title", "datetime"]
                    }
                }
            }
        ],
        tool_choice="auto"
    )

    # 处理函数调用
    assistant_message = response.choices[0].message

    if assistant_message.tool_calls:
```

```
        # 添加助手的消息（包含函数调用）
        messages.append({
            "role": "assistant",
            "content": assistant_message.content,
            "tool_calls": assistant_message.tool_calls
        })

        # 执行每个函数调用
        for tool_call in assistant_message.tool_calls:
            function_name = tool_call.function.name
            function_args = json.loads(tool_call.function.arguments)

            print(f"调用函数: {function_name}")
            print(f"参数: {function_args}")

            # 执行函数
            result = function_map[function_name](**function_args)
            print(f"结果: {result}")

            # 添加函数结果到消息
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": json.dumps(result)
            })

        # 第二次调用，让模型整合结果
        response = client.chat.completions.create(
            model="gpt-4o",
            messages=messages
        )

    return response.choices[0].message.content

# 使用示例
result = chat_with_functions("北京今天天气怎么样？")
print(result)

result = chat_with_functions("帮我给test@example.com发一封邮件，主题是'会议通知'，内容是'明天下午3点开会'")
print(result)
```

3.3.3 高级函数模式

并行函数调用：

```
# 模型可以同时调用多个函数
response = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
    tools=functions,
    tool_choice="auto"  # 模型决定调用哪些函数
)

# 处理多个并行调用
for tool_call in response.choices[0].message.tool_calls:
    # 分别执行每个函数
    result = execute_function(tool_call)
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": json.dumps(result)
    })
```

强制特定函数调用：

```
# 强制使用某个函数
response = client.chat.completions.create(
    model="gpt-4o",
    messages=messages,
```

```
    tools=functions,
    tool_choice={"type": "function", "function": {"name": "get_weather"}}
)
```

3.4 助手创建实战

图3-3：助手创建流程图

```
┌──────────────
```

```
│   │  Step 5: 轮询 Run 状态                                    │    │
│   │  ──────────────────────────────────────────────────────────────────────────────────────────────────────────  │    │
│   │                                                            │    │
│   │  while run.status in ["queued", "in_progress"]:                 │    │
│   │      run = client.beta.threads.runs.retrieve(                   │    │
│   │          thread_id=thread_id,                                   │    │
│   │          run_id=run_id                                          │    │
│   │      )                                                          │    │
│   │      time.sleep(1)  # 等待                                     │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                           ↓                                        │
│   ┌──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│
│   │  Step 6: 获取回复消息                                      │    │
│   │  ──────────────────────────────────────────────────────────────────────────────────────────────────────────  │    │
│   │                                                            │    │
│   │  messages = client.beta.threads.messages.list(                  │    │
│   │      thread_id=thread_id                                        │    │
│   │  )                                                              │    │
│   │                              ↓                                 │    │
│   │  提取 assistant 的最新回复                                     │    │
│   └──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
│
│                                                                    │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
```

3.4.1 创建Python助手的完整代码

```python
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

# ========== Step 1: 创建 Assistant ==========
assistant = client.beta.assistants.create(
    name="Python助教小Py",
    description="专业的Python编程助手，帮助用户学习和解决Python相关问题",
    instructions="""
你是一个专业的Python编程助手。

## 你的职责
- 解答Python编程问题
- 提供代码示例和最佳实践
- 帮助调试代码错误
- 解释Python概念和语法

## 回答规范
- 代码使用```python代码块```包裹
- 重要概念使用**加粗**
- 每个示例包含简短说明
- 如果不确定，明确说明
""",
    model="gpt-4o",
    tools=[
        {
            "type": "function",
            "function": {
                "name": "search_documentation",
                "description": "搜索Python官方文档获取准确信息",
                "parameters": {
                    "type": "object",
                    "properties": {
                        "query": {"type": "string", "description": "搜索关键词"}
                    },
                    "required": ["query"]
                }
            }
        }
    ]
```

```python
)

assistant_id = assistant.id
print(f"Assistant创建成功: {assistant_id}")

# ========== Step 2: 创建 Thread ==========
thread = client.beta.threads.create()
thread_id = thread.id
print(f"Thread创建成功: {thread_id}")

# ========== Step 3: 添加消息 ==========
def ask_question(question: str):
    # 添加用户消息
    client.beta.threads.messages.create(
        thread_id=thread_id,
        role="user",
        content=question
    )

    # ========== Step 4: 创建 Run ==========
    run = client.beta.threads.runs.create(
        thread_id=thread_id,
        assistant_id=assistant_id,
        instructions="请用中文回答，保持友好专业的态度。"
    )

    run_id = run.id
    print(f"Run创建成功: {run_id}")

    # ========== Step 5: 轮询状态 ==========
    import time
    while True:
        run = client.beta.threads.runs.retrieve(
            thread_id=thread_id,
            run_id=run_id
        )
        print(f"Run状态: {run.status}")

        if run.status == "completed":
            break
        elif run.status == "failed":
            print("Run执行失败")
            return None
        elif run.status == "requires_action":
            # 处理函数调用
            handle_required_actions(run, thread_id)
        else:
            time.sleep(1)

    # ========== Step 6: 获取消息 ==========
    messages = client.beta.threads.messages.list(thread_id=thread_id)

    # 打印所有消息
    print("\n=== 对话历史 ===")
    for msg in messages.data:
        role = msg.role
        content = msg.content[0].text.value
        print(f"\n[{role.upper()}]\n{content}")

    return messages.data[0].content[0].text.value

def handle_required_actions(run, thread_id):
    """处理函数调用请求"""
    if run.required_action:
        tool_calls = run.required_action.submit_tool_outputs.tool_calls

        for tool_call in tool_calls:
            if tool_call.function.name == "search_documentation":
                args = json.loads(tool_call.function.arguments)
                result = search_documentation(args["query"])
```

```
            # 提交函数结果
            client.beta.threads.runs.submit_tool_outputs(
                thread_id=thread_id,
                run_id=run.id,
                tool_outputs=[
                    {
                        "tool_call_id": tool_call.id,
                        "output": json.dumps(result)
                    }
                ]
            )

def search_documentation(query: str):
    """模拟搜索Python文档"""
    docs = {
        "list comprehension": "列表推导式是Python创建列表的简洁语法：[expr for item in iterable]",
        "decorator": "装饰器是Python中修改函数行为的强大工具，使用@符号应用"
    }
    return docs.get(query.lower(), "未找到相关文档")

# ========= 测试 =========
import json
response = ask_question("什么是列表推导式？请给个例子")
```

3.5 文件上传与知识扩展

图3-4：文件上传→向量检索→问答流程

```
┌──────────────
```

```
│   │                   │   问题向量化     │                          │   │
│   │                   │  Embedding(question)                        │   │
│   │                   └─────────┬────────┘                           │   │
│   │                             ↓                                    │   │
│   │                   ┌──────────────────────────┐                   │   │
│   │                   │   相似度检索     │                          │   │
│   │                   │                 │                           │   │
│   │                   │  ┌──────────────────────┐ │                 │   │
│   │                   │  │ Chunk 1    │ │  similarity: 0.92          │   │
│   │                   │  ├──────────────────────┤ │                 │   │
│   │                   │  │ Chunk 2    │ │  similarity: 0.85          │   │
│   │                   │  ├──────────────────────┤ │                 │   │
│   │                   │  │ Chunk 3    │ │  similarity: 0.78          │   │
│   │                   │  └──────────────────────┘ │                 │   │
│   │                   └──────────────────────────┘                   │   │
│   │                             ↓                                    │   │
│   │                 返回Top-K相关片段                                 │   │
│   └──────────────────────────────────────────────────────────────────────┘
│
│                             ↓                                        │
│   ┌──────────────────────────────────────────────────────────────────────┐
│
│   │  Phase 3: 生成回答                                                  │   │
│   │  ──────────────────────────────────────────────────────────────────────
```

```
根据用户提供的文档内容，准确回答用户的问题。
如果文档中没有相关信息，请明确说明。
""",
    model="gpt-4o",
    tools=[
        {
            "type": "file_search",
            "file_search": {
                "max_num_results": 5,
                "ranking_options": {
                    "score_threshold": 0.5
                }
            }
        }
    ]
)

# ========= 将文件关联到Assistant（可选） =========
# 如果文件是助手的知识库的一部分
assistant = client.beta.assistants.update(
    assistant.id,
    tool_resources={
        "file_search": {
            "vector_stores": [
                {
                    "vector_store_id": "vs_xxxxx"  # 可预先创建的vector store
                }
            ]
        }
    }
)

# ========= 对话时使用文件 =========
# 创建thread并添加消息
thread = client.beta.threads.create()

# 添加用户消息（可附带文件）
client.beta.threads.messages.create(
    thread_id=thread.id,
    role="user",
    content="这份文档的主要内容是什么？",
    attachments=[
        {
            "file_id": file_id,
            "tools": [{"type": "file_search"}]
        }
    ]
)

# 创建run执行
run = client.beta.threads.runs.create(
    thread_id=thread.id,
    assistant_id=assistant.id
)

# 等待完成
import time
while run.status != "completed":
    time.sleep(1)
    run = client.beta.threads.runs.retrieve(thread_id=thread.id, run_id=run.id)

# 获取结果
messages = client.beta.threads.messages.list(thread_id=thread.id)
for msg in messages.data:
    if msg.role == "assistant":
        print(msg.content[0].text.value)
```

3.5.2 创建Vector Store（知识库）

```python
# 创建Vector Store
vector_store = client.beta.vector_stores.create(
    name="产品知识库",
    expires_after={
        "anchor": "last_active",
        "days": 30
    }
)

vector_store_id = vector_store.id
print(f"Vector Store创建成功: {vector_store_id}")

# 批量上传文件
file_paths = ["doc1.pdf", "doc2.pdf", "doc3.pdf"]
batch = client.beta.vector_stores.file_batches.create(
    vector_store_id=vector_store_id,
    files=[open(path, "rb") for path in file_paths]
)

print(f"批量上传状态: {batch.status}")

# 添加单个文件
client.beta.vector_stores.files.create(
    vector_store_id=vector_store_id,
    file=open("new_doc.pdf", "rb")
)
```

3.6 自定义操作：让助手与外部API交互

3.6.1 日历助手完整实现

案例3-2：构建一个日历管理助手

```python
from openai import OpenAI
from datetime import datetime, timedelta
import json

client = OpenAI(api_key="your-api-key")

# ========== 定义日历相关函数 ==========
calendar_functions = [
    {
        "name": "create_event",
        "description": "在日历中创建新的日程事件",
        "parameters": {
            "type": "object",
            "properties": {
                "title": {
                    "type": "string",
                    "description": "日程标题"
                },
                "start_time": {
                    "type": "string",
                    "description": "开始时间，格式：YYYY-MM-DD HH:MM"
                },
                "end_time": {
                    "type": "string",
                    "description": "结束时间，格式：YYYY-MM-DD HH:MM"
                },
                "description": {
                    "type": "string",
                    "description": "日程详细描述（可选）"
                },
                "location": {
                    "type": "string",
                    "description": "会议地点（可选）"
                },
                "attendees": {
                    "type": "array",
```

```
                "items": {"type": "string"},
                "description": "参与者邮箱列表（可选）"
            }
        },
        "required": ["title", "start_time", "end_time"]
    }
},
{
    "name": "list_events",
    "description": "查询日历中的日程事件",
    "parameters": {
        "type": "object",
        "properties": {
            "start_date": {
                "type": "string",
                "description": "查询开始日期，格式：YYYY-MM-DD"
            },
            "end_date": {
                "type": "string",
                "description": "查询结束日期，格式：YYYY-MM-DD"
            }
        },
        "required": ["start_date", "end_date"]
    }
},
{
    "name": "delete_event",
    "description": "删除指定的日程事件",
    "parameters": {
        "type": "object",
        "properties": {
            "event_id": {
                "type": "string",
                "description": "要删除的事件ID"
            }
        },
        "required": ["event_id"]
    }
},
{
    "name": "find_free_slot",
    "description": "在指定时间范围内查找空闲时间槽",
    "parameters": {
        "type": "object",
        "properties": {
            "start_date": {
                "type": "string",
                "description": "开始日期，格式：YYYY-MM-DD"
            },
            "end_date": {
                "type": "string",
                "description": "结束日期，格式：YYYY-MM-DD"
            },
            "duration_minutes": {
                "type": "integer",
                "description": "需要的时间长度（分钟）",
                "default": 60
            }
        },
        "required": ["start_date", "end_date"]
    }
}
]

# ========= 模拟日历存储 =========
events_db = []

def create_event(title, start_time, end_time, description=None, location=None, attendees=None):
    """创建日程事件"""
    event = {
```

```
        "id": f"evt_{len(events_db) + 1}",
        "title": title,
        "start_time": start_time,
        "end_time": end_time,
        "description": description,
        "location": location,
        "attendees": attendees or [],
        "created_at": datetime.now().isoformat()
    }
    events_db.append(event)
    return event

def list_events(start_date, end_date):
    """查询日程"""
    filtered = []
    for event in events_db:
        event_date = event["start_time"].split()[0]
        if start_date <= event_date <= end_date:
            filtered.append(event)
    return sorted(filtered, key=lambda x: x["start_time"])

def delete_event(event_id):
    """删除日程"""
    global events_db
    for i, event in enumerate(events_db):
        if event["id"] == event_id:
            events_db.pop(i)
            return {"status": "deleted", "event_id": event_id}
    return {"status": "not_found"}

def find_free_slot(start_date, end_date, duration_minutes=60):
    """查找空闲时间槽（模拟实现）"""
    # 简单实现：返回几个建议时间
    suggestions = []
    base_date = datetime.strptime(start_date, "%Y-%m-%d")

    for day in range((datetime.strptime(end_date, "%Y-%m-%d") - base_date).days + 1):
        current = base_date + timedelta(days=day)
        suggestions.append({
            "date": current.strftime("%Y-%m-%d"),
            "slots": ["09:00-10:00", "14:00-15:00", "16:00-17:00"]
        })

    return {"suggestions": suggestions}

# ========= 创建日历助手 =========
calendar_assistant = client.beta.assistants.create(
    name="日历管理助手",
    description="专业的日程管理助手，帮助用户创建、查询和管理日历事件",
    instructions="""
你是一个专业的日历管理助手。

## 你的能力
- 创建日历事件（会议、约会、提醒等）
- 查询指定日期范围的日程
- 删除不需要的日程
- 查找空闲时间槽

## 回答规范
- 时间格式统一使用 YYYY-MM-DD HH:MM
- 创建事件后，明确告诉用户事件ID
- 查询时按时间顺序排列
- 如果用户没有指定结束时间，默认1小时
- 如果时间冲突，提醒用户
""",
    model="gpt-4o",
    tools=[{"type": "function", "function": f} for f in calendar_functions]
)

print(f"日历助手创建成功: {calendar_assistant.id}")
```

```python
# ========= 对话处理函数 =========
function_map = {
    "create_event": create_event,
    "list_events": list_events,
    "delete_event": delete_event,
    "find_free_slot": find_free_slot
}

def process_message(thread_id: str, user_message: str, assistant_id: str):
    """处理用户消息"""
    # 添加用户消息
    client.beta.threads.messages.create(
        thread_id=thread_id,
        role="user",
        content=user_message
    )

    # 创建run
    run = client.beta.threads.runs.create(
        thread_id=thread_id,
        assistant_id=assistant_id
    )

    # 处理函数调用
    while True:
        run = client.beta.threads.runs.retrieve(
            thread_id=thread_id,
            run_id=run.id
        )

        if run.status == "completed":
            break
        elif run.status == "requires_action":
            # 处理函数调用
            tool_calls = run.required_action.submit_tool_outputs.tool_calls

            outputs = []
            for tool_call in tool_calls:
                func_name = tool_call.function.name
                args = json.loads(tool_call.function.arguments)

                print(f"调用函数: {func_name}, 参数: {args}")

                # 执行函数
                result = function_map[func_name](**args)
                print(f"函数结果: {result}")

                outputs.append({
                    "tool_call_id": tool_call.id,
                    "output": json.dumps(result, ensure_ascii=False)
                })

            # 提交函数结果
            run = client.beta.threads.runs.submit_tool_outputs(
                thread_id=thread_id,
                run_id=run.id,
                tool_outputs=outputs
            )
        else:
            import time
            time.sleep(0.5)

    # 获取最终回复
    messages = client.beta.threads.messages.list(thread_id=thread_id)
    return messages.data[0].content[0].text.value

# ========= 测试日历助手 =========
# 创建线程
thread = client.beta.threads.create()
```

```python
# 测试对话
print("\n=== 测试1: 创建会议 ===")
response = process_message(
    thread.id,
    "帮我安排一个产品评审会议，明天上午10点开始，1小时后结束",
    calendar_assistant.id
)
print(f"助手回复: {response}")

print("\n=== 测试2: 查询日程 ===")
response = process_message(
    thread.id,
    "查看明天的日程安排",
    calendar_assistant.id
)
print(f"助手回复: {response}")

print("\n=== 测试3: 查找空闲时间 ===")
response = process_message(
    thread.id,
    "后天下午有什么空闲时间吗？我想开一个30分钟的会",
    calendar_assistant.id
)
print(f"助手回复: {response}")
```

3.6.2 多函数协同示例

```python
# 更复杂的多函数协同场景
complex_functions = [
    {
        "name": "get_current_weather",
        "description": "获取指定城市的当前天气",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string"}
            },
            "required": ["city"]
        }
    },
    {
        "name": "find_nearby_restaurants",
        "description": "查找附近的餐厅",
        "parameters": {
            "type": "object",
            "properties": {
                "location": {"type": "string"},
                "cuisine": {"type": "string"},
                "price_range": {
                    "type": "string",
                    "enum": ["$", "$$", "$$$", "$$$$"]
                }
            },
            "required": ["location"]
        }
    },
    {
        "name": "create_calendar_event",
        "description": "创建日历事件",
        "parameters": {
            "type": "object",
            "properties": {
                "title": {"type": "string"},
                "start_time": {"type": "string"},
                "end_time": {"type": "string"}
            },
            "required": ["title", "start_time", "end_time"]
        }
```

```
    },
    {
        "name": "send_notification",
        "description": "发送通知",
        "parameters": {
            "type": "object",
            "properties": {
                "user_id": {"type": "string"},
                "message": {"type": "string"},
                "channel": {
                    "type": "string",
                    "enum": ["email", "sms", "push"]
                }
            },
            "required": ["user_id", "message"]
        }
    }
]

# 多函数协同：自动安排外出就餐
def auto_plan_dinner():
    """
    场景：用户说"帮我安排今晚6点的餐厅约会，通知我的朋友"

    需要多函数协同：
    1. 获取天气（决定室内/室外）
    2. 查找餐厅
    3. 创建日历
    4. 发送通知
    """
    pass
```

案例实战

案例3-1：构建一个PDF文档问答助手

```
from openai import OpenAI
import json

client = OpenAI(api_key="your-api-key")

# ========= Step 1: 上传PDF文档 =========
print("Step 1: 上传PDF文档...")
pdf_file = client.files.create(
    file=open("ai_guide.pdf", "rb"),
    purpose="assistants"
)
print(f"文件ID: {pdf_file.id}")

# ========= Step 2: 创建文档问答助手 =========
print("\nStep 2: 创建文档问答助手...")
doc_assistant = client.beta.assistants.create(
    name="文档问答助手",
    instructions="""
你是一个专业的文档阅读助手。

你的职责：
1. 仔细阅读用户上传的文档
2. 准确回答用户关于文档内容的问题
3. 如果文档中没有提到相关信息，直接说明"文档中没有提及此内容"
4. 引用文档中的具体内容来支持你的回答

回答格式：
- 先给出直接回答
- 然后引用相关文档片段（使用>引用格式）
- 最后可以适当扩展解释
""",
    model="gpt-4o",
```

```python
    tools=[
        {
            "type": "file_search",
            "file_search": {
                "max_num_results": 3
            }
        }
    ],
    tool_resources={
        "file_search": {
            "vector_stores": [{
                "file_ids": [pdf_file.id]
            }]
        }
    }
)
print(f"助手ID: {doc_assistant.id}")

# ========== Step 3: 创建对话线程 ==========
thread = client.beta.threads.create()

# ========== Step 4: 定义问答函数 ==========
def ask_document(question: str):
    """向文档提问"""
    # 添加问题
    client.beta.threads.messages.create(
        thread_id=thread.id,
        role="user",
        content=question
    )

    # 执行
    run = client.beta.threads.runs.create(
        thread_id=thread.id,
        assistant_id=doc_assistant.id
    )

    # 等待完成
    import time
    while run.status != "completed":
        time.sleep(1)
        run = client.beta.threads.runs.retrieve(thread.id, run.id)

    # 获取回复
    messages = client.beta.threads.messages.list(thread.id)
    return messages.data[0].content[0].text.value

# ========== Step 5: 测试 ==========
print("\n=== 文档问答测试 ===\n")

questions = [
    "这份文档的主题是什么？",
    "AI Agent的四大核心组件是什么？",
    "如何部署本地LLM模型？",
    "这本书适合什么水平的读者？"
]

for q in questions:
    print(f"问题: {q}")
    answer = ask_document(q)
    print(f"回答:\n{answer}\n")
    print("-" * 50)
```

案例3-2：构建一个日历管理助手（完整实现）

（已在上节3.6.1中详细展示）

本章小结

本章介绍了如何构建第一个AI Agent：

1. **GPT Assistants**：OpenAI提供的Agent构建框架，支持持久线程、文件访问和自定义工具
2. **Playground使用**：通过可视化界面快速创建和测试助手
3. **Function Calling**：定义函数让Agent调用外部API，实现真正的工作能力
4. **助手创建实战**：通过Python SDK创建Python助教助手
5. **文件处理**：上传PDF文档，构建基于文档的问答系统
6. **日历助手案例**：完整实现了一个日历管理助手，支持创建、查询、删除日程

下一步：

- 第4章将深入讲解Prompt工程的高级技巧
- 第5章将介绍记忆管理系统，实现持久化交互

第一部分总结

第1-3章核心知识点

章节	主题	核心技能
第1章	AI Agent概述	六层架构、五大组件、四层演进
第2章	访问LLM	API对比、参数调优、本地部署、统一接口
第3章	构建Agent	Assistants、Function Calling、文件处理

关键概念速查

AI Agent = LLM + 工具 + 记忆 + 规划 + 自我评估

六层架构 = 用户交互层 + Agent核心层 + 推理评估层 + 记忆知识层 + 动作工具层 + LLM底座层

五大组件 = Profile + Actions/Tools + Memory/Knowledge + Reasoning/Evaluation + Planning/Feedback

Function Calling流程 = 定义函数 → 模型推理 → 本地执行 → 结果整合 → 生成回复

继续阅读

- **第4章**：Prompt工程——如何写好系统提示词
- **第5章**：Memory记忆管理——实现持久化上下文
- **第6章**：Tools工具设计——扩展Agent能力边界

本内容为《AI Agents实战指南》第1-3章，版权所有。

《AI Agents实战指南》第四章：Profile与Persona设计

4.1 什么是Profile与Persona

Profile（配置）和Persona（角色）是Agent的灵魂定义。Profile描述Agent的整体结构和工具需求，Persona则定义Agent的主要角色和特征，通常在系统提示词中体现。

【图解】Profile三大组成部分：

```
┌──────────────────────────────────────────────────────────────┐
│          Profile（配置）            │
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────────────┐  ┌──────────────────────┐  │
│  │   Persona    │  │ Demographics │  │
│  │   （角色）    │  │  （人口统计）  │  │
│  └──────────────────────┘  └──────────────────────┘  │
│  ┌──────────────────────┐                  │
│  │    Rules     │                  │
│  │   （规则）    │                  │
│  └──────────────────────┘                  │
└──────────────────────────────────────────────────────────────┘
```

Persona（角色）

定义Agent的主要功能和行为模式。例如"编程助手"、"客服"、"导师"等。

Demographics（人口统计）

描述Agent的背景特征：年龄、性别、专业领域、语言风格等。

Rules（规则）

Agent必须遵守的行为准则，如"不能透露隐私"、"回复要简洁"等。

4.2 四种人格创建方法

方法	说明	适用场景
手绘法（Handcrafted）	人工手动编写Profile描述	简单场景、快速原型
LLM辅助生成（LLM generated）	让LLM根据需求描述生成Profile	中等复杂度
数据驱动（Data generated）	从真实对话数据中提取Profile	企业级应用
进化算法（Evolutionary）	用遗传算法优化Profile参数	研究、优化

【图解】四种方法对比：

```
手绘法     ■■■■■■■■■\\\
```

4.5 实战案例：构建"专业代码审查员"Agent

```
# 带Rules的专业代码审查Agent
code_reviewer_prompt = """你是"码神"，一个严谨的代码审查专家。

【Persona】技术扎实，说话直接，喜欢给出具体改进建议
【Rules】
1. 重点检查：安全性、性能、可读性
2. 发现问题要指出具体行号
3. 给出可运行的改进代码
4. 评价用"优秀/良好/需改进"三档
5. 不说"还不错"这种模糊话"""

def review_code(code_snippet):
    response = client.chat.completions.create(
        model="gpt-4",
        messages=[
            {"role": "system", "content": code_reviewer_prompt},
            {"role": "user", "content": f"请审查这段代码：\n{code_snippet}"}
        ]
    )
    return response.choices[0].message.content
```

本章小结

要点	内容
Profile定义	Agent的基础角色配置
Persona	核心角色定义
Demographics	背景特征描述
Rules	行为约束规则
创建方法	手绘/LLM/数据/进化
一致性维护	Prompt锁定+示例强化

《AI Agents实战指南》第五章：Actions与工具调用

5.1 Actions的三维度理解

Actions（动作）是Agent执行任务的核心能力。从三个维度理解：

维度	说明	示例
Action Target（目标）	定义任务完成、探索或沟通	"查天气"、"写代码"
Action Effect（效果）	行动如何影响环境和状态	天气API返回温度数据
Action Generation（生成）	动作如何被创建和调用	手动注册/从记忆召回

【图解】Actions三维度：

```
Action Target ──→ 定义"做什么"
      ↓
Action Effect ──→ 影响"产生什么结果"
      ↓
Action Generation ──→ 决定"如何生成"
```

5.2 语义函数 vs 原生函数

类型	定义	特点
语义函数	LLM驱动的函数，用自然语言描述	灵活、可解释、依赖LLM理解
原生函数	传统程序性函数，代码实现	精确、可测试、不依赖LLM

【图解】语义函数 vs 原生函数：

```
语义函数                    原生函数
┌──────────────────┐              ┌──────────────────┐
│ "计算平均值" │              │ def avg() │
│  (LLM理解) │              │ (代码)   │
└──────────────────┘              └──────────────────┘
      ↓                              ↓
  可能理解错误                  精确执行
```

5.3 Function Calling机制

OpenAI的Function Calling让LLM能自动解析结构化参数调用外部工具。

【图解】Function Calling流程:

```
用户请求 → LLM识别意图 → 提取参数 → 调用函数 → 返回结果 → LLM汇总
     ↓           ↓            ↓
  "查北京天气"  "weather"  {city:"北京"}
```

完整示例:

```python
import json
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

# 定义工具函数
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "查询城市天气",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description": "城市名称"}
                },
                "required": ["city"]
            }
        }
    }
]

def get_weather(city):
    """模拟天气查询"""
    return f"{city}今天晴天，25度"

# 对话循环
messages = [{"role": "user", "content": "北京今天天气怎么样？"}]

response = client.chat.completions.create(
    model="gpt-4",
    messages=messages,
    tools=tools
)

# 解析函数调用
if response.choices[0].message.tool_calls:
    tool_call = response.choices[0].message.tool_calls[0]
    function_name = tool_call.function.name
    arguments = json.loads(tool_call.function.arguments)

    if function_name == "get_weather":
        result = get_weather(arguments["city"])
        print(f"查询结果：{result}")
```

5.4 工具注册与调用

【图解】工具注册时序图:

```
开发者 ──→ 注册工具 ──→ Agent系统
              ↓
         工具描述注册
              ↓
用户请求 ──→ Agent匹配 ──→ 调用工具 ──→ 返回结果
```

5.5 实战案例：构建天气查询工具

```python
import json
from datetime import datetime

# 工具定义
weather_tool = {
    "name": "weather查询",
    "description": "查询指定城市的当前天气",
    "parameters": {
        "type": "object",
        "properties": {
            "city": {
                "type": "string",
                "description": "城市名称，支持中英文"
            },
            "unit": {
                "type": "string",
                "enum": ["celsius", "fahrenheit"],
                "description": "温度单位"
            }
        },
        "required": ["city"]
    }
}

def weather(city: str, unit: str = "celsius") -> dict:
    """天气查询工具（示例实现）"""
    # 实际项目中调用真实天气API
    return {
        "city": city,
        "weather": "晴天",
        "temperature": 25 if unit == "celsius" else 77,
        "unit": unit,
        "time": datetime.now().isoformat()
    }

# 使用示例
result = weather("北京", "celsius")
print(result)
# {'city': '北京', 'weather': '晴天', 'temperature': 25, 'unit': 'celsius', ...}
```

5.6 工具调用失败处理

策略	说明	代码示例
重试	自动重试N次	`for i in range(3): try: ...`
降级	失败时使用简化方案	`if failed: use_fallback()`
报错	无法处理时返回明确错误	`raise ValueError()`

```python
def safe_call_tool(func, *args, max_retries=3):
    """安全的工具调用封装"""
    for attempt in range(max_retries):
        try:
            return func(*args)
        except Exception as e:
            if attempt == max_retries - 1:
                return {"error": str(e), "fallback": True}
```

```
        continue
    return {"error": "Max retries exceeded"}
```

本章小结

要点	内容
Actions三维度	Target（目标）、Effect（效果）、Generation（生成）
语义函数	LLM驱动的自然语言函数
原生函数	代码实现的确切函数
Function Calling	自动解析参数调用工具
工具注册	描述→发现→调用→结果
失败处理	重试、降级、报错

《AI Agents实战指南》第六章：记忆与知识系统

6.1 AI Agent的记忆类型体系

人类有多种记忆类型，AI Agent的设计也借鉴了这些概念：

记忆类型	人类对应	AI Agent对应
感官记忆	原始感官输入	用户原始输入
短期记忆	当前会话上下文	Context Window
长期记忆	跨会话持久存储	向量数据库/知识库
语义记忆	抽象概念事实	知识图谱
情景记忆	具体经历事件	对话历史摘要
程序记忆	技能操作流程	Agent的规划能力

【图解】人类记忆 vs AI Agent记忆对比：

```
人类记忆                    AI Agent记忆
┌──────────────┐            ┌──────────────┐
│感官记忆  │            │原始输入  │
└──────┬───────┘            └──────┬───────┘
       ↓                           ↓
┌──────────────┐            ┌──────────────┐
│短期记忆  │            │Context   │
└──────┬───────┘            └──────┬───────┘
       ↓                           ↓
┌──────────────┐            ┌──────────────┐
│长期记忆  │            │向量库    │
└──────────────┘            └──────────────┘
```

6.2 RAG（检索增强生成）两阶段架构

RAG是给Agent"外部知识"的核心技术，分两阶段：

索引阶段（Build Index）

```
文档 → 分块(Chunking) → 嵌入(Embedding) → 向量数据库
```

查询阶段（Query）

```
查询 → 嵌入 → 向量相似度搜索 → 上下文 → LLM生成
```

【图解】RAG两阶段架构：

```
【索引阶段】
文档 → 清洗 → 分块 → 向量化 → 向量库存储
```

```
【查询阶段】
用户查询 → 向量化 → 相似度匹配 → Top-K相关块 → 上下文组装 → LLM生成答案
```

6.3 向量数据库对比

数据库	类型	开源	云服务	适用规模
Chroma DB	向量	✅	❌	小-中规模，快速原型
Pinecone	向量	❌	✅	大规模，企业级
Weaviate	向量+知识图谱	✅	✅	中-大规模，混合检索
Milvus	向量	✅	✅	超大规模

【图解】向量数据库选型矩阵：

```
                 小规模        大规模
        ┌──────────────────┬──────────────────┐
开源    │  Chroma DB       │   Milvus         │
        ├──────────────────┼──────────────────┤
云服务  │  Pinecone        │  Weaviate        │
        └──────────────────┴──────────────────┘
```

6.4 实战案例：使用Chroma DB构建本地知识库

```python
# 安装：pip install chromadb openai

import chromadb
from chromadb.config import Settings
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

# 1. 初始化Chroma DB
chroma_client = chromadb.Client(Settings(
    persist_directory="./chroma_db",
    anonymized_telemetry=False
))

# 2. 创建集合
collection = chroma_client.create_collection("knowledge_base")

# 3. 添加文档
documents = [
    "苹果公司成立于1976年，总部位于加州库比蒂诺",
    "乔布斯是苹果公司的联合创始人",
    "iPhone是苹果2007年推出的智能手机产品"
]

# 4. 向量化并存储
collection.add(
    documents=documents,
    ids=[f"doc_{i}" for i in range(len(documents))]
)

# 5. 查询
results = collection.query(
    query_texts=["苹果创始人是谁？"],
    n_results=2
)

for doc in results['documents'][0]:
    print(f"相关文档：{doc}")
```

6.5 记忆压缩技术

当上下文窗口不足时，需要对记忆进行压缩：

K-means聚类 + LLM总结：

```
对话历史 → K-means聚类 → 每类摘要 → 压缩后记忆
```

【图解】记忆压缩流程：

```
长期记忆（原始）
     ↓ K-means聚类
┌─────────┐  ┌─────────┐  ┌─────────┐
│类1 │  │类2 │  │类3 │  ← 按主题分组
└─────────┘  └─────────┘  └─────────┘

     ↓ LLM总结
┌──────────────────────────────────┐
│  压缩后记忆摘要    │
└──────────────────────────────────┘
```

本章小结

要点	内容
记忆类型	感官/短期/长期/语义/情景/程序
RAG两阶段	索引（文档→向量）+ 查询（匹配→生成）
向量数据库	Chroma/Pinecone/Weaviate/Milvus
记忆压缩	K-means聚类+LLM总结

《AI Agents实战指南》第七章：推理与评估机制

7.1 推理技术谱系

从简单提示到复杂推理，技术在不断进化：

【图解】推理技术演进谱系：

```
直接提示 ──→ Few-shot ──→ CoT ──→ Self-Consistency ──→ ToT
  │           │           │           │                │
  ↓           ↓           ↓           ↓                ↓
无示例       给示例      展示思维    多路径投票        树状探索
```

技术	全称	特点
Zero-shot	零样本	无需示例，直接推理
Few-shot	少样本	提供1-5个示例引导
CoT	Chain of Thought	展示推理步骤
Self-Consistency	自我一致性	多路径生成，一致性投票
ToT	Tree of Thought	树状探索，breadth/depth搜索

7.2 Chain of Thought（CoT）链式推理

核心思想：让LLM展示推理过程，而不是直接给答案。

标准CoT格式：

```
问题：小明有10个苹果，给了小红3个，又买了5个，现在有几个？
思考：先算给了小红3个后剩10-3=7个，然后加上买的5个，7+5=12个
答案：12个
```

Zero-shot CoT：用"Let's think step by step"触发推理。

```
# Zero-shot CoT示例
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "user", "content": "小明的公司去年营收100万，今年比去年增长30%，明年预计再增长20%，明年的营收是多少？\nLet's think step by step."}
    ]
```

```
)
print(response.choices[0].message.content)
```

输出示例:

```
Step 1: 计算今年营收 = 100万 × (1 + 30%) = 130万
Step 2: 计算明年营收 = 130万 × (1 + 20%) = 156万
答案：明年营收预计156万
```

7.3 Tree of Thought（ToT）树状推理

核心思想：探索多条解决路径，评估后选择最佳。

【图解】ToT树状探索:

```
                 问题
              /  |  \
            /    |    \
        路径1  路径2  路径3
         /\     |     /\
       子1 子2  子3  子4 子5
        |     |     |
       评估A  评估B  评估C
        \     |     /
           最优路径 → 答案
```

代码示例:

```
def tree_of_thought(question, options):
    """简化版ToT实现"""
    results = []
    for option in options:
        # 每个分支进行CoT推理
        response = client.chat.completions.create(
            model="gpt-4",
            messages=[
                {"role": "user", "content": f"{question}\n方案：{option}\n请分析这个方案的优缺点。"}
            ]
        )
        results.append({
            "option": option,
            "analysis": response.choices[0].message.content
        })

    # 评估选择
    evaluation = client.chat.completions.create(
        model="gpt-4",
        messages=[
            {"role": "user", "content": f"请从以下方案中选择最佳：\n{results}\n选择并说明理由。"}
        ]
    )
    return evaluation.choices[0].message.content
```

7.4 Self-Consistency（自我一致性）

核心思想：生成多个推理路径，选择最一致的答案。

【图解】Self-Consistency多路径投票:

```
问题 ——→ 生成N个推理路径 ——→ 每个路径产生答案
            ↓
        答案投票统计
            ↓
        最一致的答案输出
```

7.5 实战案例：CoT数学推理

```python
def math_reasoning(question):
    """带CoT的数学推理助手"""

    cot_prompt = f"""请仔细分析这个数学问题，列出详细步骤。

问题：{question}

格式要求：
1. 明确列出已知条件
2. 写出计算公式
3. 逐步计算
4. 最终给出答案

Let's think step by step."""

    response = client.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": cot_prompt}]
    )
    return response.choices[0].message.content

# 测试
print(math_reasoning("一个水池有进水管和出水管，进水管8分钟注满，出水管12分钟排空，两管同时打开，几分钟注满？"))
```

本章小结

要点	内容
推理技术谱系	Zero-shot → Few-shot → CoT → Self-Consistency → ToT
CoT链式推理	展示推理步骤，增强可信度
Zero-shot CoT	"Let's think step by step"触发
ToT树状推理	多路径探索，Breadth/Depth搜索
Self-Consistency	多路径投票，最一致答案

《AI Agents实战指南》第八章：多Agent系统基础

8.1 为什么需要多Agent

单个Agent能力有限，复杂问题需要多个专业化Agent协作。

【图解】单Agent vs 多Agent：

```
单Agent                        多Agent
┌──────────────────────┐              ┌──────────────────────┐
│    单一角色    │              │  Controller  │
│    串行执行    │              └──────────┬───────────┘
│   能力有上限   │                   /     \
              │           ┌──────────┐  ┌──────────┐
              │           │Agent A  │  │Agent B│
              │           │(专业1)  │  │(专业2)│
              │           └──────────┘  └──────────┘
              │              \     /
              │            ┌──────────────────┐
              │            │    协作输出    │
              └────────────────────┴──────────────────────┘
```

多Agent核心优势：

1. **并行处理** - 多个Agent同时工作
2. **相互反馈** - Agent间互相评估改进
3. **减少错误** - 交叉验证降低失误

8.2 Agent间通信机制

消息线程模式：

```
Agent A  ──消息──→  共享历史  ←──消息──  Agent B
                        ↓
                  Controller协调
```

8.3 控制器/代理模式

【图解】控制器模式架构:

```
用户请求
    ↓
┌──────────────────────┐
│ Controller  │ ← 协调者角色
│  （控制器）  │
└──────────┬──────────┘
      ↓ 分派任务
┌──────────┐ ┌──────────┐ ┌──────────┐
│ Worker │ │ Worker │ │ Worker │
│  A    │ │  B    │ │  C    │
└────┬────┘ └────┬────┘ └────┬────┘
   ↓        ↓        ↓
  结果     结果      结果
   \       |       /
    └──────┴──────┘
          ↓
      Controller汇总
```

8.4 AutoGen多Agent模式

模式	说明	适用场景
Two-Agent Chat	两个Agent对话	简单协作
Group Chat	多Agent共享对话	复杂协作
Nested Chat	Agent调用Agent	嵌套任务

【图解】AutoGen Group Chat:

```
┌──────────────────────────────────────────────────────────┐
│        GroupChatManager          │
│        （群组聊天管理器）          │
└──────────┬──────────────────────────────────────┬──────────┘
      ↓                  ↓
┌──────────────┐        ┌──────────────┐
│ Agent A │        │ Agent B │
│(发送者) │        │(接收者) │
└──────────────┘        └──────────────┘
     ↘              ↙
      ↘  Group Chat  ↙
       ↘   共享对话  ↙
        ↙          ↘
         └──────────┘
```

8.5 CrewAI多Agent模式

模式	说明
Sequential	依次执行，任务依赖
Hierarchical	管理层→执行层
Parallel	多Agent同时执行

【图解】CrewAI Sequential流程:

```
Task A → Task B → Task C → 最终结果
  ↑        ↑        ↑
Agent1   Agent2   Agent3
```

【图解】CrewAI Hierarchical:

```
┌──────────────────────┐
│ Manager      │  ← 管理层
│  Agent       │
└──────────────┬───────┘
       ↓ 分派
┌──────────────┐  ┌──────────────┐
│Worker  │  │Worker  │
│Agent1  │  │Agent2  │
└──────────────┘  └──────────────┘
```

8.6 实战案例：AutoGen双Agent对话

```python
# 安装：pip install autogen

from autogen import ConversableAgent, Agent

# 创建助手Agent
assistant = ConversableAgent(
    name="assistant",
    system_message="你是一个有帮助的助手。",
    llm_config={"model": "gpt-4"}
)

# 创建用户代理
user_proxy = ConversableAgent(
    name="user_proxy",
    human_input_mode="NEVER"
)

# 开始对话
result = user_proxy.initiate_chat(
    assistant,
    message="请用Python写一个快速排序函数"
)
print(result)
```

8.7 实战案例：CrewAI新闻采编团队

```python
# 安装：pip install crewai

from crewai import Agent, Task, Crew, Process

# 定义Agent角色
researcher = Agent(
    role="记者",
    goal="收集最新科技新闻",
    backstory="资深科技记者，善于发现热点"
)

editor = Agent(
    role="编辑",
    goal="审核并修改新闻稿",
    backstory="资深编辑，注重准确性和可读性"
)

publisher = Agent(
    role="发布者",
    goal="发布新闻到平台",
    backstory="专业运营，熟悉各平台规则"
)

# 定义任务
tasks = [
    Task(description="收集今天AI领域最新动态", agent=researcher),
    Task(description="审核新闻稿并修改", agent=editor),
    Task(description="发布到微信公众号", agent=publisher)
```

```
]

# 创建团队
crew = Crew(
    agents=[researcher, editor, publisher],
    tasks=tasks,
    process=Process.sequential
)

# 启动
result = crew.kickoff()
```

本章小结

要点	内容
多Agent价值	并行、专业化、交叉验证
通信机制	消息线程、共享历史
控制器模式	Controller协调多个Worker
AutoGen模式	Two-Agent/Group/Nested Chat
CrewAI模式	Sequential/Hierarchical/Parallel

《AI Agents实战指南》第九章：AutoGen高级应用

9.1 AutoGen核心概念

概念	说明
ConversableAgent	可对话的Agent基类
GroupChat	群组聊天，多Agent共享对话
GroupChatManager	协调群组聊天

9.2 Two-Agent Chat深度实战

【图解】Two-Agent Chat时序:

```
User Proxy ──→ [发起对话] ──→ Assistant
    ↑                            |
    |                            ↓
    └──────────── [接收回复] ←──────────────────────┘
```

```python
from autogen import ConversableAgent

# 专家Agent
expert = ConversableAgent(
    name="expert",
    system_message="你是一个Python专家，用简洁代码解释概念。",
    llm_config={"model": "gpt-4"}
)

# 用户代理
user = ConversableAgent(
    name="user",
    human_input_mode="NEVER"
)

# 发起对话
result = user.initiate_chat(
    expert,
    message="解释什么是装饰器"
)
```

9.3 Group Chat模式

【图解】Group Chat消息流转:

```
Agent A ──→ GroupChatManager ←── Agent B
    │                              │
    └──────────── 共享对话历史 ────────────┘
```

```python
from autogen import GroupChat, GroupChatManager, ConversableAgent

# 创建多个Agent
coder = ConversableAgent(name="coder", system_message="你是Python程序员")
reviewer = ConversableAgent(name="reviewer", system_message="你是代码审查员")
tester = ConversableAgent(name="tester", system_message="你负责写测试")

# 创建群组聊天
group_chat = GroupChat(
    agents=[coder, reviewer, tester],
    messages=[],
    max_round=5
)

# 创建管理器
manager = GroupChatManager(groupchat=group_chat)

# 启动群组讨论
result = coder.initiate_chat(
    manager,
    message="写一个计算器类并测试"
)
```

9.4 Nested Chat（嵌套对话）

Agent可以将另一个Agent作为工具调用:

```python
# Agent A 调用 Agent B 作为工具
agent_a = ConversableAgent(name="A", ...)
agent_b = ConversableAgent(name="B", ...)

# A将B作为工具注册
agent_a.register_for_execution(name="B")(agent_b)
```

9.5 AgentOps可观测性

监控Agent运行状态的关键指标:

指标	说明
Token消耗	每次调用的Token用量
延迟	响应时间
错误率	调用失败比例
成本	计算成本

【图解】AgentOps监控仪表盘:

```
┌──────────────────────────────────────────────┐
│        AgentOps Dashboard                    │
├──────────────────────────────────────────────┤
│ Token消耗   ■■■
```

要点	内容
ConversableAgent	可对话Agent基类
Two-Agent Chat	双Agent协作
Group Chat	多Agent共享对话
Nested Chat	Agent嵌套调用
AgentOps	可观测性监控

《AI Agents实战指南》第十章：CrewAI企业级应用

10.1 CrewAI核心概念

概念	说明
Agent	role/goal/backstory/allow_delegation
Task	任务描述和预期输出
Crew	Agent团队
Process	任务执行流程

【图解】CrewAI概念关系:

```
Crew（团队）
  ├── Agent（角色）
  │     ├── role: 角色名
  │     ├── goal: 目标
  │     ├── backstory: 背景故事
  │     └── allow_delegation: 是否允许委派
  │
  └── Process（执行流程）
        ├── Sequential（串行）
        ├── Hierarchical（层级）
        └── Parallel（并行）
```

10.2 Sequential Tasks（串行执行）

```python
from crewai import Agent, Task, Crew, Process

# 创建Agent
writer = Agent(role="Writer", goal="撰写产品文案", backstory="资深文案")
reviewer = Agent(role="Reviewer", goal="审核文案质量", backstory="专业审核")

# 创建任务
task1 = Task(description="写一篇AI产品推广文案", agent=writer)
task2 = Task(description="审核并修改文案", agent=reviewer)

# 创建Crew（串行执行）
crew = Crew(
    agents=[writer, reviewer],
    tasks=[task1, task2],
    process=Process.sequential
)

result = crew.kickoff()
```

10.3 Hierarchical Process（层级执行）

```python
# 管理层Agent
manager = Agent(
    role="Manager",
    goal="协调团队完成任务",
    backstory="经验丰富的管理者",
    allow_delegation=True  # 允许委派任务
```

```
)

# 执行层Agent
worker1 = Agent(role="Researcher", goal="收集信息")
worker2 = Agent(role="Analyst", goal="分析数据")

# 层级流程会自动创建
crew = Crew(
    agents=[manager, worker1, worker2],
    process=Process.hierarchical
)
```

10.4 Parallel Tasks（并行执行）

```
# 三个Agent同时工作
agent_a = Agent(role="A", goal="任务A")
agent_b = Agent(role="B", goal="任务B")
agent_c = Agent(role="C", goal="任务C")

crew = Crew(
    agents=[agent_a, agent_b, agent_c],
    process=Process.parallel
)
```

10.5 CrewAI vs AutoGen对比

特性	CrewAI	AutoGen
学习曲线	低	中等
多Agent支持	优秀	优秀
工具调用	良好	优秀
企业级功能	强	中等
适用场景	企业流程自动化	研究原型

【图解】选型决策树:

```
项目类型?
    ├── 企业流程 → CrewAI
    ├── 研究原型 → AutoGen
    ├── 快速验证 → CrewAI
    └── 深度定制 → AutoGen
```

10.6 实战案例：简历筛选团队

```
from crewai import Agent, Task, Crew, Process

hr = Agent(role="HR", goal="初筛简历", backstory="招聘专员")
tech_screen = Agent(role="Tech", goal="技术评估", backstory="技术面试官")
final_review = Agent(role="Manager", goal="最终决策", backstory="部门经理")

tasks = [
    Task(description="从100份简历中筛选出符合基本条件的", agent=hr),
    Task(description="对通过初筛的简历进行技术能力评估", agent=tech_screen),
    Task(description="综合评估给出最终录用建议", agent=final_review)
]

crew = Crew(
    agents=[hr, tech_screen, final_review],
    tasks=tasks,
    process=Process.sequential
)

result = crew.kickoff()
```

本章小结

要点	内容
Agent定义	role/goal/backstory
Sequential	串行任务执行
Hierarchical	管理层委派执行层
Parallel	并行任务执行
CrewAI vs AutoGen	场景选择指南

《AI Agents实战指南》第十一章：行为树与Agent编排

11.1 行为树起源

行为树（Behavior Tree）起源于游戏AI，现已扩展到Agent编排领域。

【图解】行为树演进：

```
游戏AI行为树 ──────→ Agent行为树(ABT)
    ↓                    ↓
NPC AI控制          多Agent编排
```

核心思想：用树状结构组织Agent的决策逻辑。

11.2 ABT核心节点类型

节点类型	符号	说明
Selector（选择器）	⌥	类似"或"，尝试直到成功
Sequence（序列）	→	类似"与"，全部成功才成功
Condition（条件）	?	判断是否执行分支
Action（动作）	▶	执行具体任务
Decorator（装饰器）	◇	修改子节点行为
Parallel（并行）	⏸	同时执行多个子节点

【图解】ABT节点全览：

```
                [Root]
                   |
              [Selector]
             /         \
       [Sequence]      [Decorator]
       /    |    \          |
    [?]   [▶]   [▶]     [Retry]
     |      |      |          |
   条件   动作1  动作2     重试
```

11.3 Blackboard模式

节点间共享状态的通信机制：

【图解】Blackboard模式：

```
┌──────────────────────────────────────────────────────────┐
│         Blackboard              │
│  ┌──────────────────────────────────────────────┐     │
│  │ shared_state = {        │     │
│  │   "user_name": "张三", │     │
│  │   "last_action": "查询" │     │
│  │ }                       │     │
│  └──────────────────────────────────────────────┘     │
└──────────────────────────────────────────────────────────┘
     ↑ 写入           ↑ 读取
 ┌──────────┴──────────┐     ┌──────────┴──────────┐
```

```
│Node A    │      │Node B    │
└──────────────────┘      └──────────────────┘
```

11.4 ABT优势

优势	说明
模块化	节点独立，可复用
可扩展	容易添加新节点
可视化	树状结构直观
调试友好	易于追踪执行路径

11.5 实战案例：客服Agent行为树

```python
# 使用py_trees库构建行为树
import py_trees

# 创建行为树节点
root = py_trees.composites.Selector(name="Root", memory=False)

# 问候 Sequence
greeting_sequence = py_trees.composites.Sequence(name="Greeting", memory=False)
greeting_sequence.add_child(py_trees.decorators恒定.Choice("CheckTime"))
greeting_sequence.add_child(py_trees.actions.Execute("SayHello"))

# 查询处理
query_sequence = py_trees.composites.Sequence(name="Query", memory=False)
query_sequence.add_child(py_trees.actions.Execute("CollectQuery"))
query_sequence.add_child(py_trees.actions.Execute("ProcessQuery"))
query_sequence.add_child(py_trees.actions.Execute("ProvideAnswer"))

root.add_child(greeting_sequence)
root.add_child(query_sequence)
root.add_child(py_trees.actions.Execute("DefaultResponse"))

# 执行行为树
py_trees.trees_tick(root)
```

本章小结

要点	内容
行为树起源	游戏AI → Agent编排
核心节点	Selector/Sequence/Condition/Action/Decorator/Parallel
Blackboard	节点间共享状态
ABT优势	模块化、可扩展、可视化

《AI Agents实战指南》第十二章：Semantic Kernel与插件系统

12.1 Semantic Kernel定位

Microsoft的Agent开发框架，统一管理原生函数和语义函数。

【图解】Semantic Kernel架构：

```
┌──────────────────────────────────────────────────────────────┐
│          Application Layer          │
├──────────────────────────────────────────────────────────────┤
│         Semantic Kernel             │
│   ┌──────────────────────────────────────────────────┐      │
│   │      Kernel (核心)          │     │
│   │   ┌──────────────┐  ┌──────────────┐    │     │
│   │   │ Planner │  │ Memory │    │     │
```

```
│   │   └──────────────┘  └──────────────┘    │     │
│   └──────────────────────────────────────────────┘     │
├──────────────────────────────────────────────────────────┤
│        Plugins / Skills            │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐    │
│   │Native    │  │Semantic│  │Core      │    │
│   │Functions│  │Functions│  │Skills    │    │
│   └──────────────┘  └──────────────┘  └──────────────┘    │
└──────────────────────────────────────────────────────────┘
```

12.2 语义函数 vs 原生函数

类型	实现方式	特点
原生函数	C#/Python代码	精确、可测试
语义函数	自然语言描述	灵活、依赖LLM

12.3 实战案例：电影推荐插件

```python
# Semantic Kernel Python版示例
from semantic_kernel import Kernel
from semantic_kernel.plugins import NativePlugin

kernel = Kernel()

# 注册原生函数插件
class MoviePlugin(NativePlugin):
    @NativeFunction
    def recommend_movie(self, genre: str) -> str:
        """根据类型推荐电影"""
        recommendations = {
            "科幻": "星际穿越",
            "喜剧": "你好，李焕英",
            "悬疑": "看不见的客人"
        }
        return recommendations.get(genre, "流浪地球")

kernel.register_plugin(MoviePlugin())

# 使用
result = kernel.run("推荐一部科幻电影", plugin="MoviePlugin")
print(result)
```

12.4 Semantic Kernel vs LangChain vs AutoGen对比

框架	厂商	优势	劣势
Semantic Kernel	Microsoft	企业级、Azure集成	学习曲线
LangChain	LangChain Inc.	RAG强大	复杂、文档不足
AutoGen	Microsoft	多Agent协作	偏向研究

本章小结

要点	内容
Semantic Kernel	Microsoft的Agent框架
原生函数	代码实现的确切函数
语义函数	LLM驱动的自然语言函数
框架对比	SK vs LC vs AutoGen

《AI Agents实战指南》第十三章：Prompt Flow与系统性评估

13.1 Prompt Flow定位

Microsoft的系统性Prompt评估优化工具。

【图解】Prompt Flow流程:

```
构建Prompt → 迭代评估 → 批量评估 → Rubric评分 → 对比变体
    ↓          ↓          ↓          ↓
  编写模板    单次测试   大规模测试   A/B测试
```

13.2 Rubric（评分标准）

定义评估Prompt输出的标准和尺度:

```yaml
# rubric_example.yaml
rubric:
  name: "客服质量评估"
  criteria:
    - name: "礼貌性"
      score_range: [1, 5]
      description: "是否使用礼貌用语"
    - name: "准确性"
      score_range: [1, 5]
      description: "是否正确回答问题"
    - name: "完整性"
      score_range: [1, 5]
      description: "是否完整提供信息"
```

13.3 Grounding（落地/对齐）

评估输出与预期标准的对齐程度:

【图解】Grounding流程:

```
LLM输出 → 与标准对比 → Grounding分数
                            ↓
                        改进建议
```

13.4 实战案例：Prompt优化A/B测试

```python
# Prompt Flow简化版实现
variants = {
    "A": "你是一个客服，请礼貌回答用户问题。",
    "B": "你是一个客服专家，5年经验，擅长解决技术问题。请专业且友好地回答。"
}

def evaluate_prompt(prompt, test_cases):
    scores = []
    for case in test_cases:
        response = client.chat.completions.create(
            model="gpt-4",
            messages=[
                {"role": "system", "content": prompt},
                {"role": "user", "content": case["input"]}
            ]
        )
        # 评估响应质量
        score = evaluate_response(response, case["expected"])
        scores.append(score)
    return sum(scores) / len(scores)

# 对比测试
score_a = evaluate_prompt(variants["A"], test_cases)
score_b = evaluate_prompt(variants["B"], test_cases)

print(f"版本A得分: {score_a}")
```

```
print(f"版本B得分：{score_b}")
print(f"最佳版本：{'A' if score_a > score_b else 'B'}")
```

本章小结

要点	内容
Prompt Flow	系统性Prompt评估工具
Rubric	评分标准定义
Grounding	输出与标准对齐
A/B测试	多版本对比优化

《AI Agents实战指南》第十四章：前沿主题深度探索

14.1 多Agent通信协议标准化

未来的多Agent系统需要标准化通信协议：

协议	说明
Agent Communication Protocol	Agent间消息格式
Agent Discovery Protocol	Agent发现机制
Agent Trust Framework	Agent间信任体系

14.2 Agent安全与伦理

【图解】Agent安全风险分类：

```
Agent安全风险
    ├── 工具误用
    ├── Prompt注入
    ├── 无限循环
    ├── 数据泄露
    └── 权限滥用
```

防护措施：

1. 权限控制 - 最小权限原则
2. 输入验证 - 防止恶意指令
3. 熔断机制 - 防止无限循环
4. 审计日志 - 记录所有操作

14.3 长期记忆架构

【图解】多会话记忆管理：

```
会话1 ──→ 记忆压缩 ──┐
会话2 ──→ 记忆压缩 ──┼──→ 长期记忆库 ──→ 当前会话
会话3 ──→ 记忆压缩 ──┘          ↑
                              检索
```

14.4 多模态Agent

整合视觉、语音等多种感知能力：

【图解】多模态Agent架构：

```
┌──────────────────┐
│  文本输入  │──→ ┌──────────────────┐
└──────────────────┘    │  Agent Core  │
                        └──────┬───────┘
┌──────────────────┐
│  图像输入  │──→ ┌──────────┴──────────┐
└──────────────────┘    │ Multi-Modal │
┌──────────────────┐    │   Fusion    │
```

```
│   语音输入   │──→ └──────────────────────────┘
└──────────────────────────┘
```

14.5 自主Agent的收敛性

确保Agent不偏离目标：

策略	说明
目标检查点	定期检查是否接近目标
路径回溯	偏离时回退到上一步
超时机制	超过时间强制结束
迭代上限	最大迭代次数限制

本章小结

要点	内容
通信协议	多Agent标准化
安全伦理	风险分类与防护
长期记忆	多会话管理
多模态	视觉+语音+文本
收敛性	目标追踪机制

《AI Agents实战指南》第十五章：企业落地实战

15.1 企业Agent平台选型指南

【图解】选型决策树：

```
项目需求？
    │
    ├─── 快速验证 → CrewAI
    │
    ├─── 企业流程自动化
    │       │
    │       ├─── 已有Azure → Semantic Kernel
    │       └─── 无偏好 → CrewAI
    │
    ├─── 研究/原型 → AutoGen
    │
    └─── 复杂RAG → LangChain
```

15.2 AgentOps企业级可观测性

【图解】AgentOps监控架构：

```
┌──────────────────────────────────────────────────────────┐
│          AgentOps Dashboard          │
├──────────────────────────────────────────────────────────┤
│  实时监控        成本分析      告警    │
│  ┌──────────┐        ┌──────────┐      ┌────────┐  │
│  │Token │        │ $12 │      │ ⚠️ │  │
│  │ 78% │        │ 成本 │      │异常│  │
│  └──────────┘        └──────────┘      └────────┘  │
└──────────────────────────────────────────────────────────┘
```

15.3 成本控制策略

策略	说明
Token监控	实时追踪消耗
缓存优化	重复查询利用缓存

模型降级	简单问题用小模型
批处理	批量请求降低单位成本

15.4 企业知识库RAG落地

```
# 企业知识库RAG架构
class EnterpriseRAG:
    def __init__(self):
        self.embedding_model = "text-embedding-ada-002"
        self.vector_db = "Pinecone"  # 企业级向量数据库
        self.llm = "gpt-4"

    def ingest_documents(self, documents):
        """文档摄入"""
        for doc in documents:
            chunks = self.chunk_document(doc)
            embeddings = self.embed(chunks)
            self.vector_db.upsert(embeddings)

    def query(self, question):
        """查询"""
        query_embedding = self.embed([question])
        results = self.vector_db.search(query_embedding)
        context = self.assemble_context(results)
        return self.llm.generate(f"基于上下文回答：{question}\n\n{context}")
```

15.5 实战案例：企业客服Agent

```
# 企业客服Agent完整方案
from autogen import ConversableAgent

customer_service = ConversableAgent(
    name="客服",
    system_message="""你是企业智能客服助手。

【角色】专业、友好、高效
【能力】
- 解答产品相关问题
- 处理常见问题
- 引导用户操作
- 收集反馈意见

【规则】
- 首次响应在30秒内
- 复杂问题转人工
- 遵循数据隐私规范""",
    llm_config={
        "model": "gpt-4",
        "temperature": 0.7,
        "max_tokens": 500
    }
)

user_proxy = ConversableAgent(
    name="user",
    human_input_mode="NEVER"
)

result = user_proxy.initiate_chat(
    customer_service,
    message="我想咨询产品的价格"
)
```

15.6 行业落地案例

行业	应用场景	效果
金融	智能投顾、风控	提升效率60%
医疗	病历整理、辅助诊断	减少医生工作量40%
电商	智能客服、选品	提升转化率25%
客服	智能问答、工单处理	降低人力成本30%

本章小结

要点	内容
平台选型	根据场景选择AutoGen/CrewAI/SK
AgentOps	企业级可观测性
成本控制	Token监控、缓存、模型降级
安全加固	权限控制、审计日志
行业落地	金融、医疗、电商、客服

附录A：工具与平台速查表

A.1 主流LLM提供商对比

提供商	模型	API方式	函数调用	中国区访问
OpenAI	GPT-4o, GPT-4	REST API	原生	需代理
Azure OpenAI	GPT-4o, GPT-4	REST API	原生	企业可用
Anthropic	Claude 3.5	REST API	内部规划	需代理
Google	Gemini 1.5	REST API	原生	部分可用
Groq	Llama, Mixtral	REST API	支持	可用
LM Studio	开源模型	Local API	取决于模型	本地

A.2 Agent开发框架对比

框架	厂商	多Agent	工具调用	RAG	学习曲线
AutoGen	Microsoft	优秀	优秀	一般	中等
CrewAI	CrewAI Inc.	优秀	良好	一般	低
Semantic Kernel	Microsoft	一般	优秀	优秀	中等
LangChain	LangChain Inc.	良好	良好	优秀	高

A.3 向量数据库对比

数据库	开源	云服务	适用规模
Chroma DB	是	否	小-中
Pinecone	否	是	大
Weaviate	是	是	中-大
Milvus	是	是	超大

附录B：核心术语表（中英对照）

中文	English	一句话定义
智能体	Agent	能自主行动代表用户完成任务的AI实体
自主代理	Autonomous Agent	可独立规划和执行任务的Agent
多智能体系统	Multi-Agent System	多个专业化Agent协作的系统

大语言模型	LLM	基于Transformer的大规模语言模型
检索增强生成	RAG	结合检索和生成的架构
嵌入	Embedding	将文本转换为向量表示
链式推理	CoT	展示推理步骤的提示技术
树状推理	ToT	树状探索多路径解决方案
行为树	Behavior Tree	分层任务编排结构
函数调用	Function Calling	LLM调用外部工具的机制
提示流	Prompt Flow	系统性Prompt评估工具
向量数据库	Vector Database	存储和检索向量嵌入的数据库

附录C：推荐学习路径

C.1 开发者学习路径（0→1→生产）

```
阶段1（入门，1-2周）
  → 第1章 AI Agent概述
  → 第2章 访问大语言模型
  → 第3章 构建你的第一个Agent
  → 实战：构建简单问答助手

阶段2（单Agent开发，2-3周）
  → 第4章 Profile与Persona设计
  → 第5章 Actions与工具调用
  → 第6章 记忆与知识系统
  → 第7章 推理与评估机制
  → 实战：构建具备工具和记忆的Agent

阶段3（多Agent系统，2-3周）
  → 第8章 多Agent系统基础
  → 第9章 AutoGen高级应用
  → 第10章 CrewAI企业级应用
  → 实战：构建多Agent协作系统

阶段4（高级编排，2-3周）
  → 第11章 行为树与Agent编排
  → 第12章 Semantic Kernel
  → 第13章 Prompt Flow
  → 实战：构建生产级Agent系统

阶段5（落地生产，2-3周）
  → 第14章 前沿主题探索
  → 第15章 企业落地实战
  → 实战：企业级Agent项目实施
```